

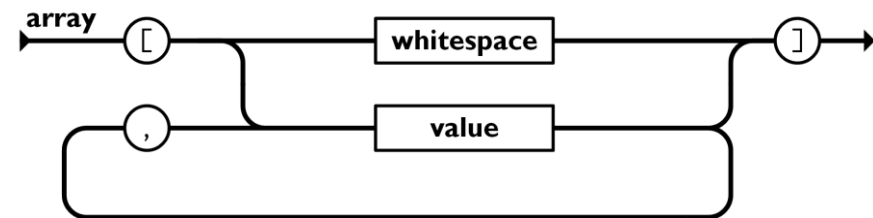
Aggregate Modeling for Document DB

JSON: syntax

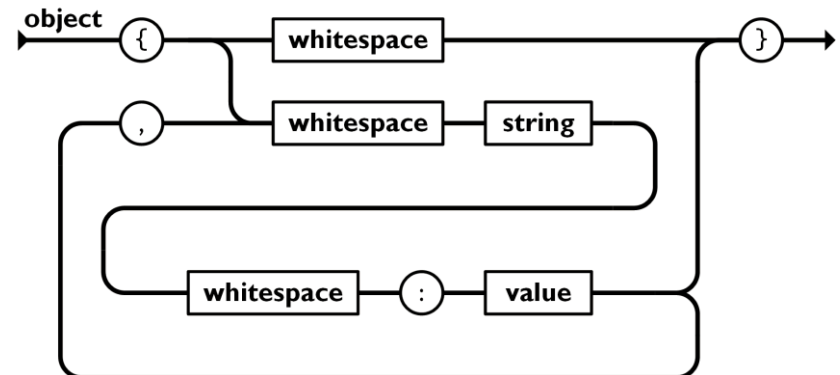
String	"apple" "\u00c4pfel\n" " "
Number	3 1.4 -1.5e3
Boolean	true false
Null	null
Array	["home", "wooden", 1, 3.14]
Objects	{ "id": 1, "name": "A wooden door", "tags": ["home", "wooden"] }

A JSON value can be one of: String, Number, Boolean, Null, Object, Array.

An array is an ordered collection of values.



An object is an unordered collection of key/value pairs



Document

Field Name (Key)

Field Value

```
{
  "_id": ObjectId("5037ee4a1084eb3fffeef7228"),
  "info": {
    "nome": "Enrico",
    "dataNascita" : ISODate("1988-08-04T20:42:00.000Z")
  },
  "interessi": ["Calcetto", "Viaggi", "Serie TV"],
  "didattica": [{
    "corso": "Big Data",
    "datore": "Università di Bologna"
  }, {
    "corso": "Introduzione ai sistemi di
    "datore": "FITSTIC"
  }]
}
```

Field Values

Scalar (Int, Boolean, String, Date, ...)

Document (Embedding or Nesting)

Array of JSON objects

Sub document {}

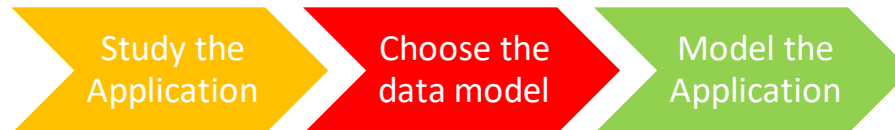
List []

One size does not fit all!

- ❑ For more than 40 years the only available data model has been the relational one
 - ✓ Applications with different needs/features have been implemented on the same model
 - ✓ Performance was limited



- ❑ The need to support applications with strict performance constraints and huge data sets led to the emergence of NoSQL DBMSs
 - ✓ Each model has different features, specific to workloads
 - ✓ The designer must know the main features of the different data models



Building Aggregates

An *aggregate* is a group of tightly coupled concepts to be handled as a document.

Defining an aggregate require:

1. Identify the set of covered concepts
2. Define the organization of the document, that is defining the primary concepts and its embeddings

Nonetheless, understanding the business domain is mandatory the starting point

Embedding vs Referencing

Designing aggregates basically means finding the right tradeoff between nesting and referencing

Nesting means storing into a main concept related concepts that are identified through relationships

- Determines larger documents
- May reduce the number of collections and objects to be accessed
- Increase redundancies if we want to optimize several different queries

Referencing means storing concepts separately, relationships are implemented by storing the IDs of the referenced concepts.

- Determines smaller documents
- Requires more collections and objects to be accessed
- Reduce redundancies

Referencing vs Nesting

Referencing

```
CUSTOMER = {  
  _id: "joe",  
  Name: "joe pesci",  
  Contacts: "Main str, 45"  
}
```

```
ORDER = {  
  _id: 1,  
  IdCus: "joe",  
  SaleDate: "14/03/2024",  
  TotalPrice: 23500  
}
```

Nesting (Embedding)

```
CUSTOMER = {  
  _id: "joe",  
  Name: "joe pesci",  
  Contacts: "Main str, 45",  
  Orders: [{ _id: 1,  
              SaleDate: "14/03/2024",  
              TotalPrice: 23500 },  
            { _id: 2,  
              SaleDate: "15/03/2024",  
              TotalPrice: 12600 }  
          ]  
}
```

Referencing vs Nesting

Referencing

```
CUSTOMER = {  
  _id: "joe",  
  Name: "joe pesci",  
  Contacts: "Main str, 45"  
}
```

```
ORDER = {  
  _id: 1,  
  IdCus: "joe",  
  SaleDate: "14/03/2024",  
  TotalPrice: 23500  
}
```

Nesting (Embedding)

```
CUSTOMER = {  
  _id: "joe",  
  Name: "joe pesci",  
  Contacts: "Main str, 45",  
  Orders: [{ _id: 1,  
             SaleDate: "14/03/2024",  
             TotalPrice: 23500 },  
            { _id: 2,  
             SaleDate: "15/03/2024",  
             TotalPrice: 12600 }  
  ]  
}
```

A reference is not a relational foreign key: no constraints on the domain values are defined

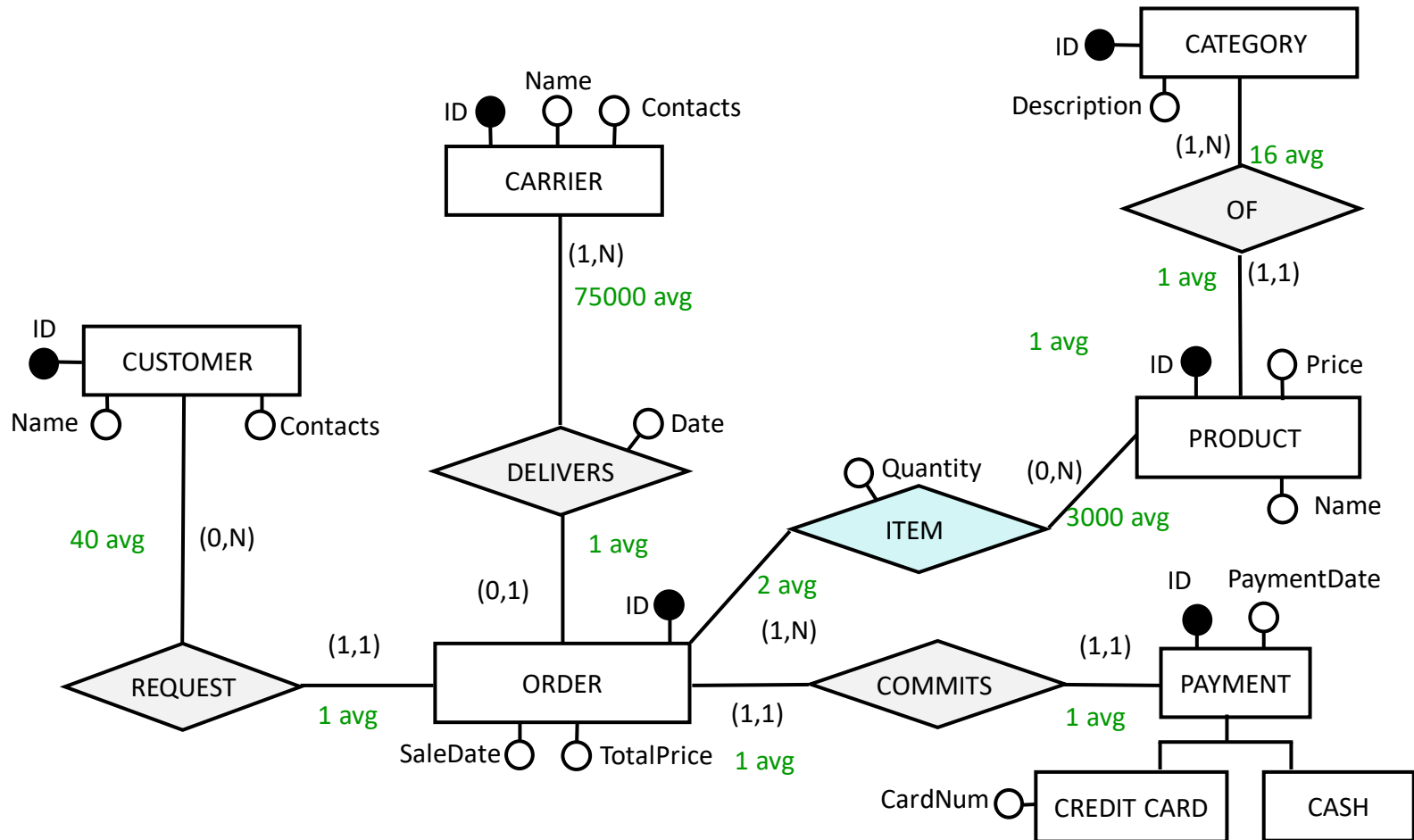
A Methodological Approach

Aggregate modeling builds aggregates of concepts rather than single concepts (i.e. entity and relationships)

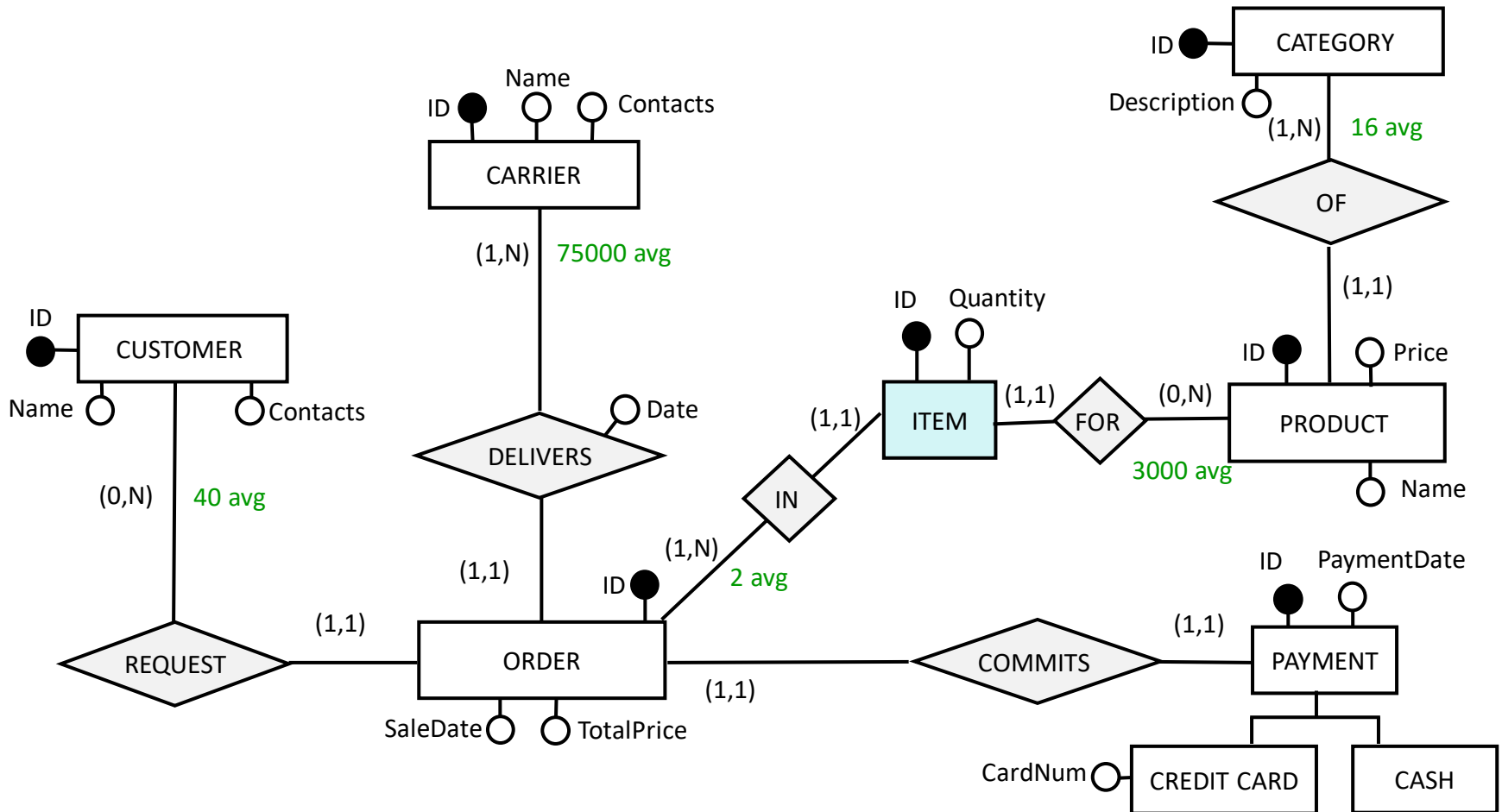
Nonetheless, understanding the business domain is mandatory the starting point

- 1) Build the E/R for the domain
- 2) Map the workload on the E/R
- 3) Find the optimal set of aggregates such that:
 - All the E/R concepts are covered at least once
 - The workload cost is optimized
 - The overall size of the collections is $< DBSize$
 - Each aggregate object size is $< ObjectSize$

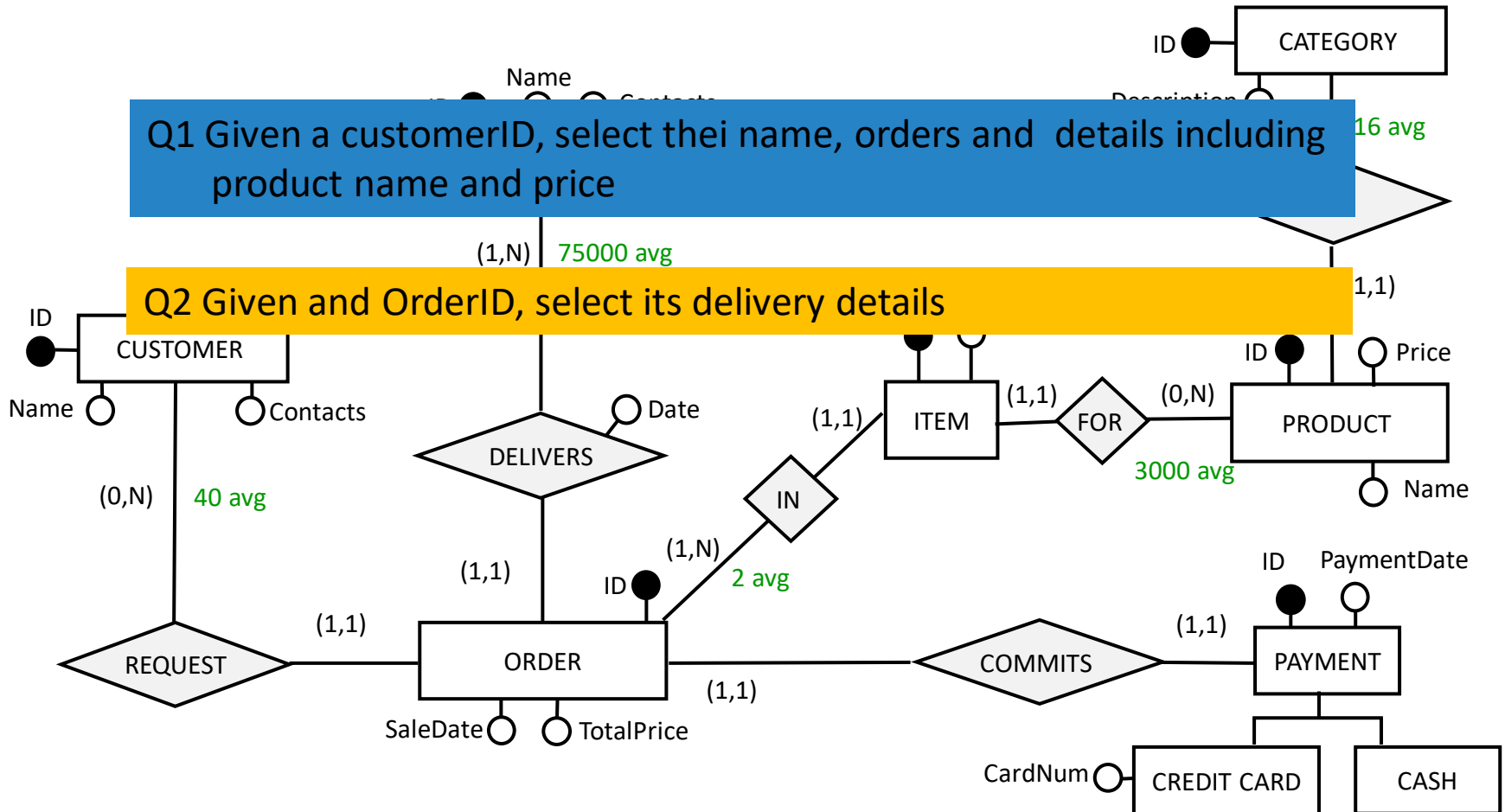
Case study



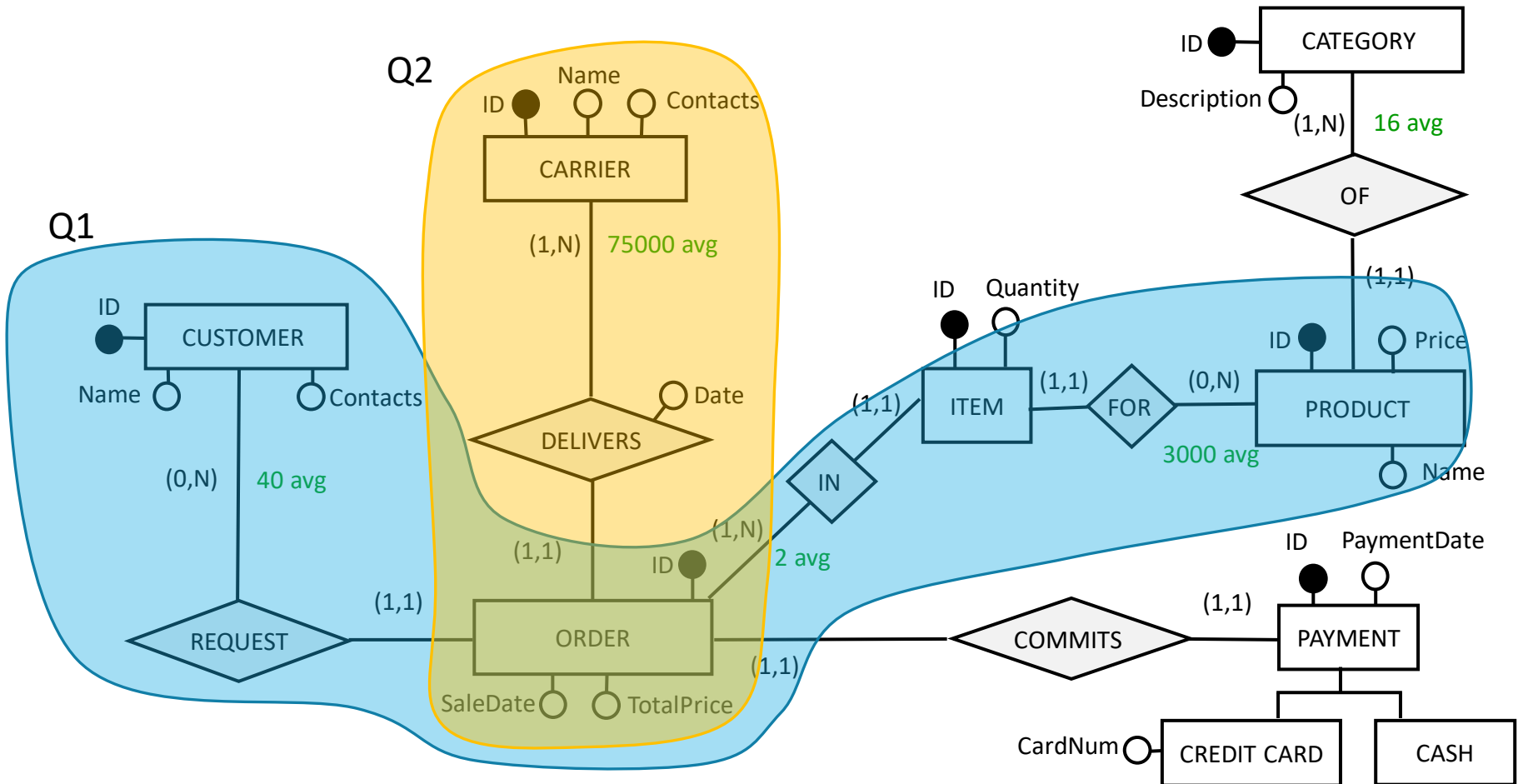
Case study: reification



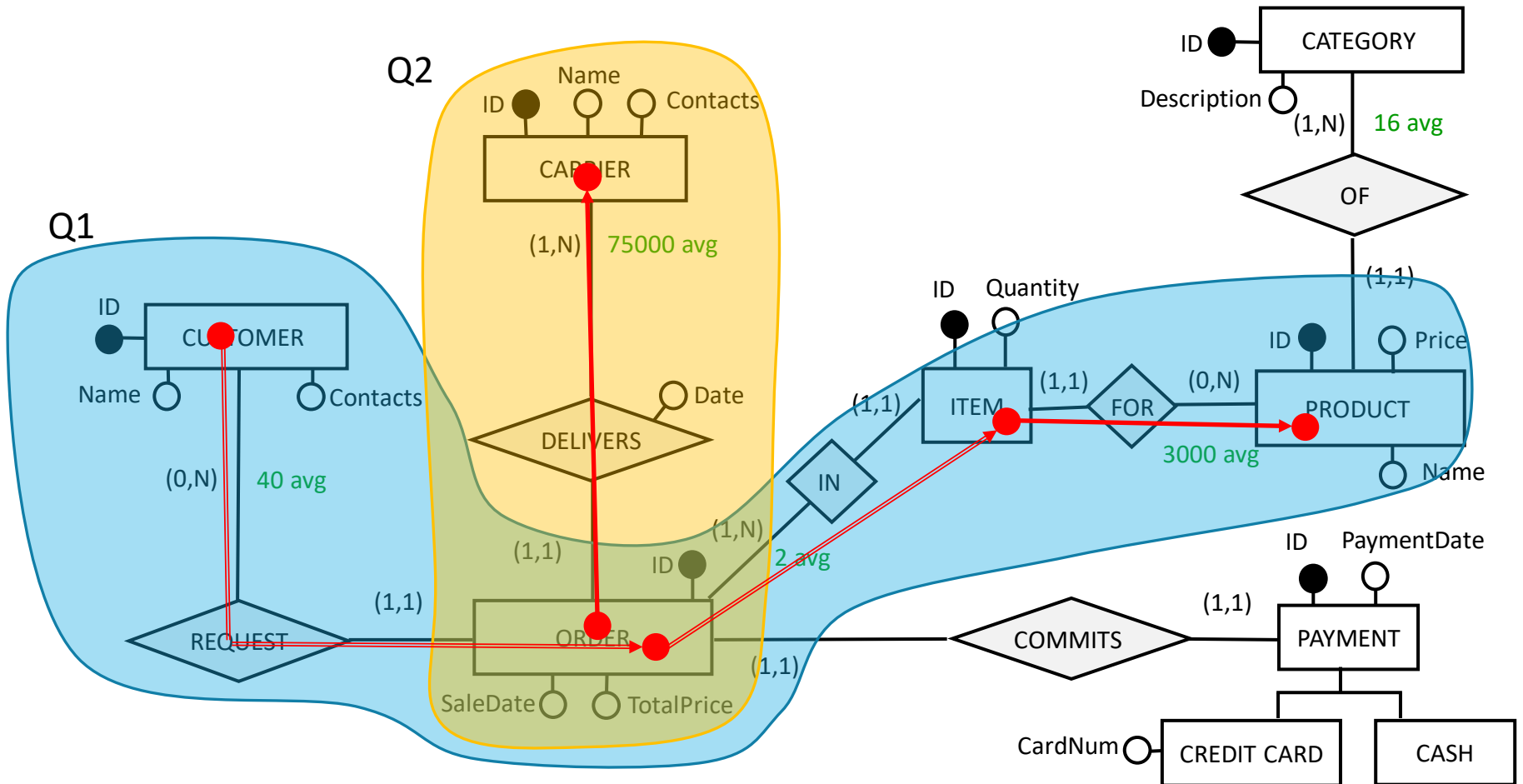
Case study: workload



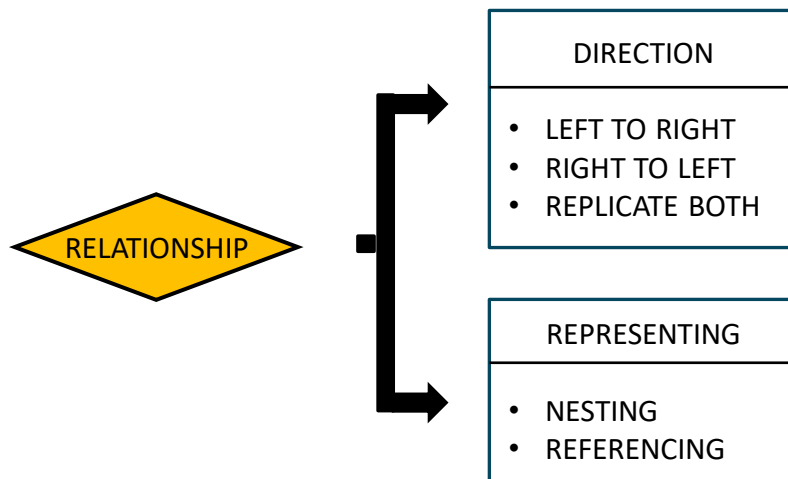
Building Aggregates



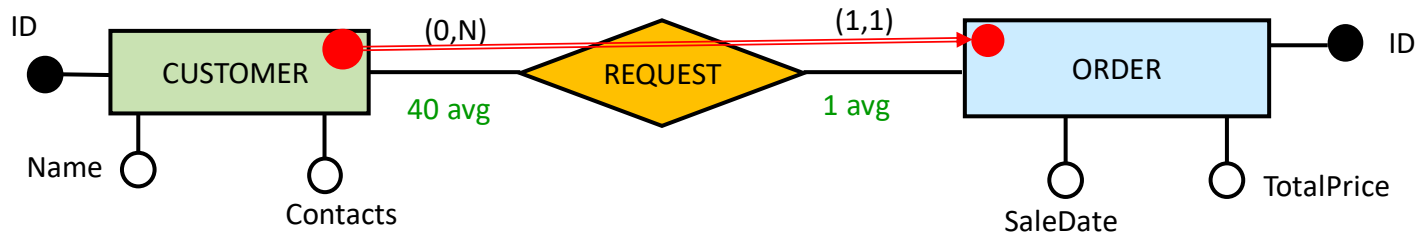
Building Aggregates



Design Solution for Relationships



One-to-many: Embedding

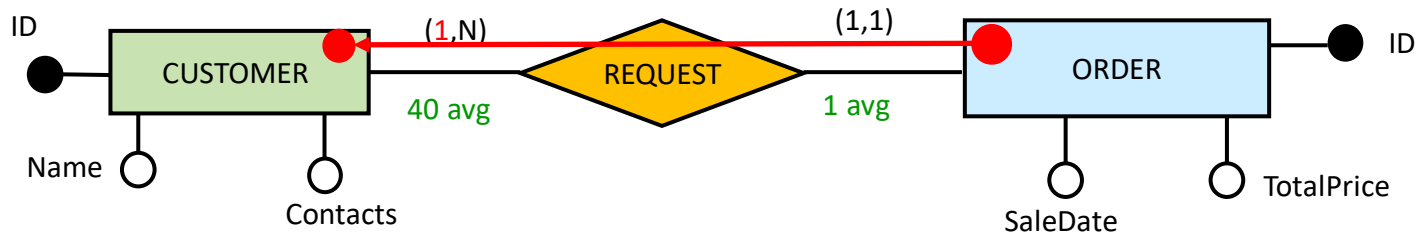


Nesting ([embedding](#)), Left→Right (many-to-one)

Customer =

```
_id,  
Name,  
Contacts,  
Orders: [{IdOrder,  
          TotalPrice,  
          SaleDate}, ...]
```


One-to-many: Flattening

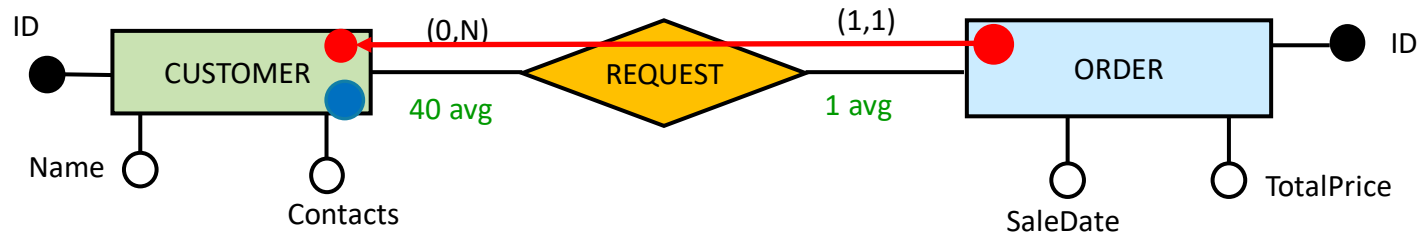


Nesting (**flattening**), Right $\rightarrow$ Left (one-to-many)

Order =

```
_id,  
SalePrice,  
TotalPrice  
Customer: { IdCust,  
            Name,  
            Contacts }
```

One-to-many



Customer =

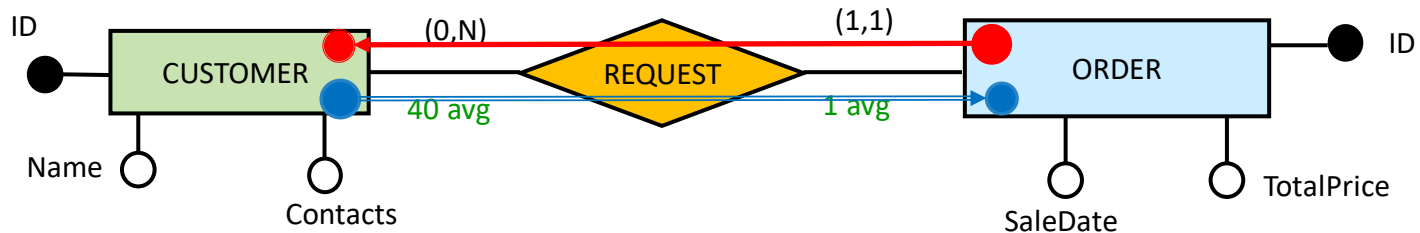
```
_id,  
Name,  
Contacts
```

Order =

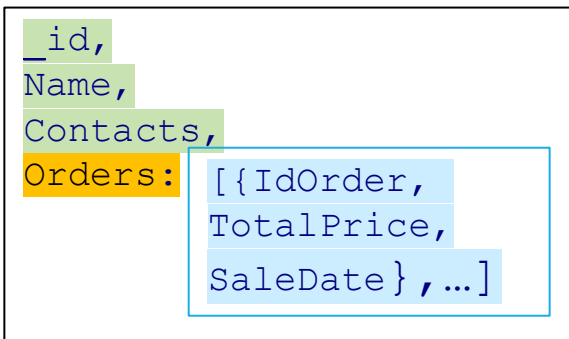
```
_id,  
SalePrice,  
TotalPrice  
Customer: { IdCust,  
            Name,  
            Contacts }
```

Flattening (one-to-many) + Replica

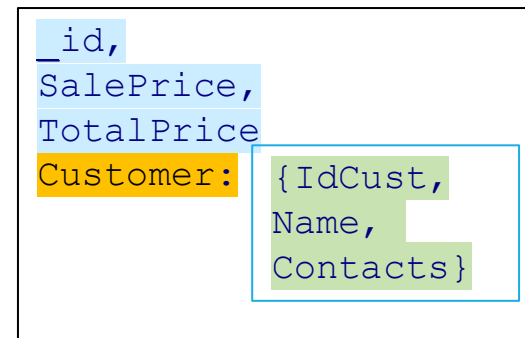
One-to-many: Double Nesting



Customer =

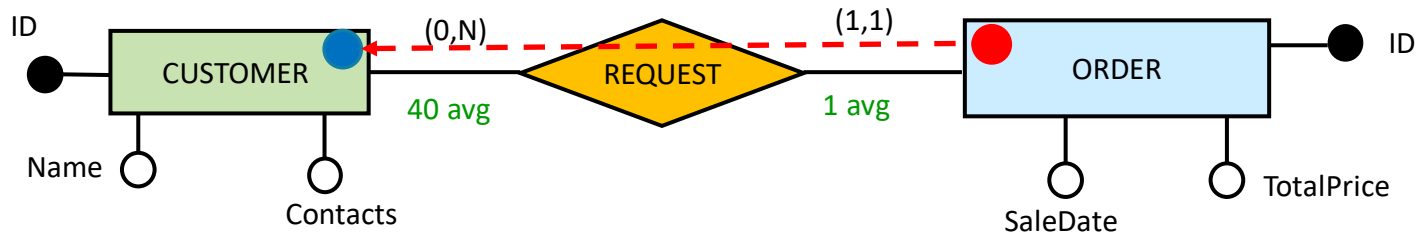


Order =



Nesting, Replicate both

One-to-many: Referencing



Referencing, Right → Left (one-to-many)

Customer =

```
_id,  
Name,  
Contacts
```

Order =

```
_id,  
SalePrice,  
TotalPrice,  
idCustomer
```

Referencing, Left to right (Many to one) or Replicate both: they are possible, but can be neglected as less efficient

Design Pattern

A **design pattern** is a generic, reusable and accepted solution to a common and recurring design problem [wikipedia]

Patterns are reusable units of knowledge:

- Approximation
- Attribute
- Bucket
- Computed
- Document Versioning
- Extended Reference
- Outlier
- Pre-allocation
- Polymorphic
- Schema Versioning
- Subset
- Tree

Patterns

Approximation
Attribute
Bucket
Computed
Document Versioning
Extended Reference
Outlier
Preallocated
Polymorphic
Schema Versioning
Subset
Tree and Graph

Use Case Categories

	Catalog	Content Management	Internet of Things	Mobile	Personalization	Real-Time Analytics	Single View
Approximation	✓		✓	✓		✓	
Attribute	✓	✓					✓
Bucket			✓			✓	
Computed	✓		✓	✓	✓	✓	✓
Document Versioning	✓	✓			✓		✓
Extended Reference	✓			✓		✓	
Outlier			✓	✓	✓		
Preallocated			✓			✓	
Polymorphic	✓	✓		✓			✓
Schema Versioning	✓	✓	✓	✓	✓	✓	✓
Subset	✓	✓		✓	✓		
Tree and Graph	✓	✓					

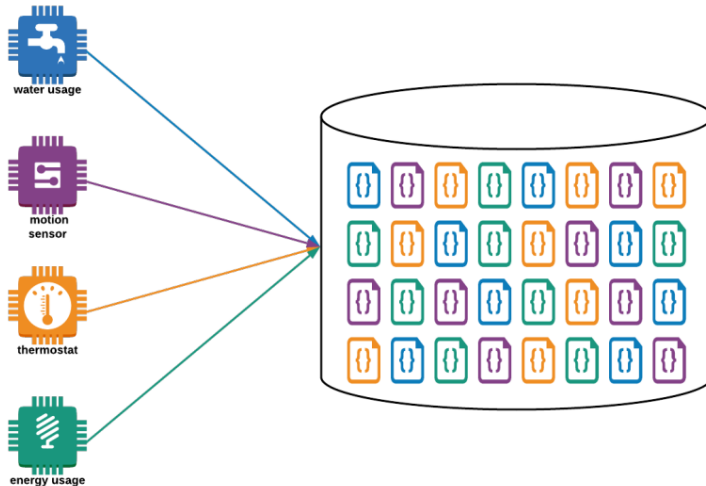
Bucket pattern

Bucketing refers to the division of documents into groups.

It is an intermediate solution between the rigidity of Embedding and the flexibility of Referencing

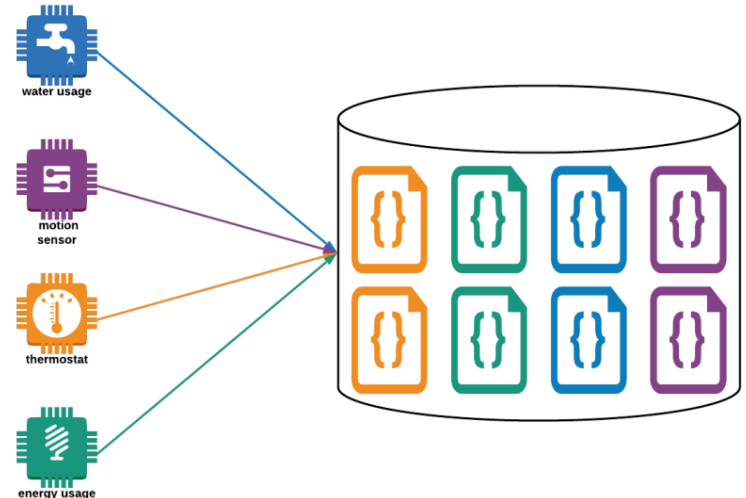
Tabular Approach

New document for each sensor reading

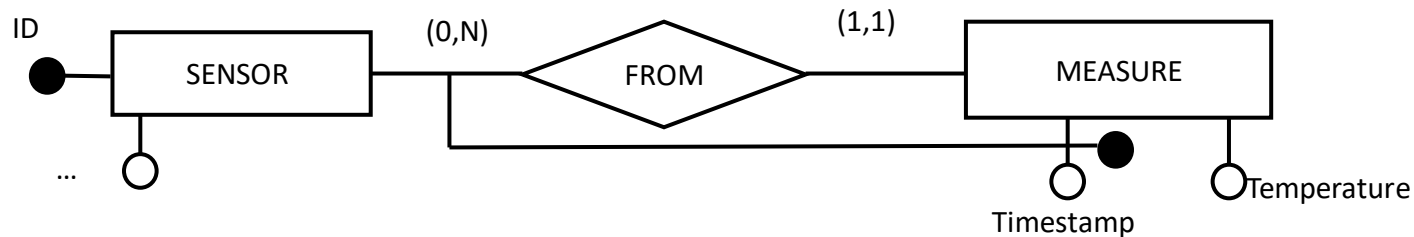


Document Approach

New document per time unit per sensor



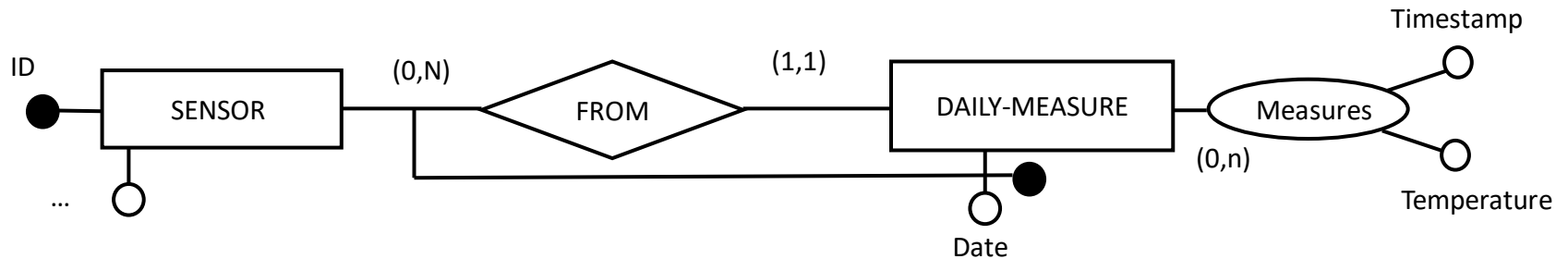
Bucket pattern - Example



```
measures = [{
  _id: 1,
  SensorID: 12345,
  Timestamp: "2019-01-31T10:00:00",
  Temperature: 40
},
{
  _id: 2,
  SensorID: 12345,
  Timestamp: "2019-01-01T10:01:00"),
},...] ... //squillion of documents
```

IoT: Measurements
from Sensors

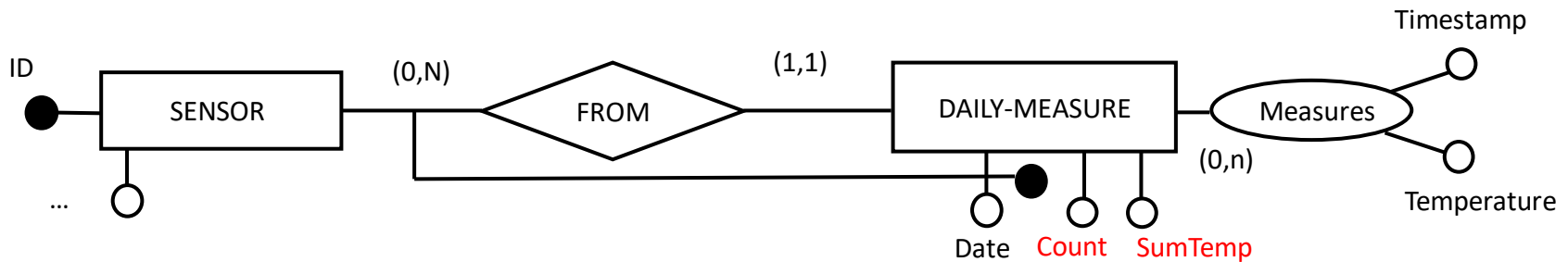
Bucket pattern - Example



```
daily_measures = [{  
  _id: 1,  
  SensorID: 12345,  
  Date: "2019-01-31",  
  Measurements: [ {  
    Timestamp: "2019-01-31T10:00:00",  
    Temperature: 40},  
    {  
    Timestamp: "2019-01-31T10:01:00.000",  
    Temperature: 38},  
    ...  
  }], ...  
}]
```

Bucketing by
time intervals

Bucket + Computed Pattern



```
daily_measures = [{
  _id: 1,
  SensorID: 12345,
  Date: "2019-01-31",
  Measurements: [ {
    Timestamp: "2019-01-31T10:00:00",
    Temperature: 40}, ... ],
  Count: 42,
  SumTemp: 2413
}, ...]
```

SumTemp = SUM(temperature)
Count = COUNT(temperature)

When the bucket includes pre-computed summary data (**derived data**) it is named Computed Pattern.

Bucket pattern: summary

Pros

- Reduces the overall number of documents in a collection
- Improves index performance
- Can simplify data access by leveraging pre-aggregation

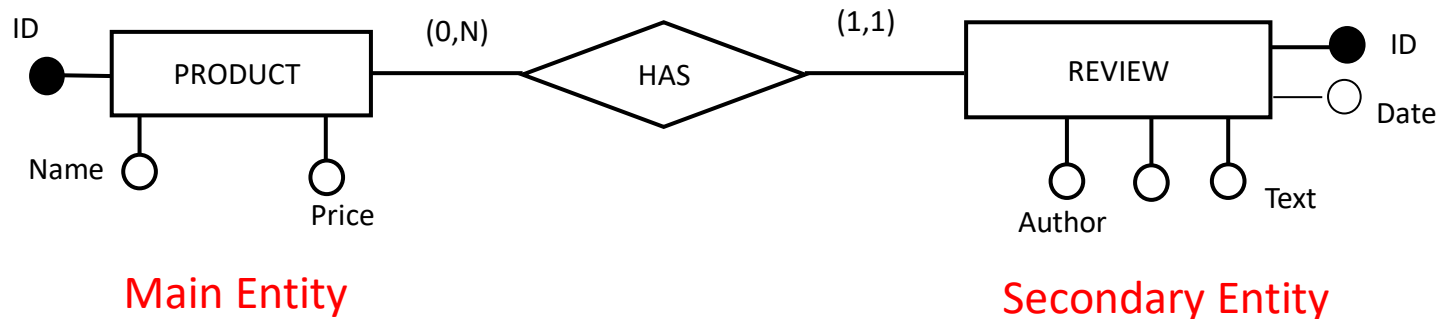
Guidelines

- Useful when needing to manage streaming data

Use cases

- Time-series
- Banking transactions
- Real-time analytics
- Internet of Things (IoT)

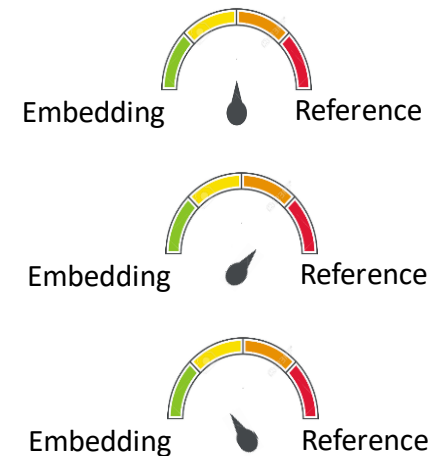
Mixing Embedding and Referencing



Mixed (Extended reference Pattern)

Reference with Embedding (Subset Pattern)

Embedding with Reference (Outlier Pattern)



Mixing Embedding and Referencing

Mixed (Extended reference Pattern) :

- Instead of embedding all secondary entity, you embed frequently accessed *fields* only (**Redundancy**).

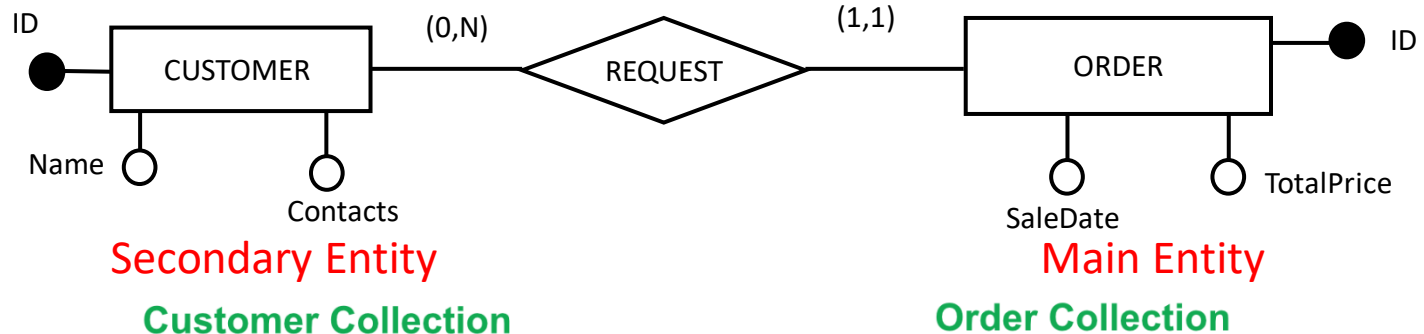
Reference with Embedding (Subset Pattern) : embedding a subset of the secondary *entity's* documents (the rest are linked):

- The secondary entity is linked (**referencing**)
- The most frequently used data is **also embedded** in the main entity (**Redundancy**)

Embedding with Reference (Outlier Pattern): inverse of the previous one, used when the number of documents to be embedded is high only in a few cases (**outliers**):

- The secondary entity is **embedded** in the main entity
- In cases where the array of embedded documents is too long, only the first N elements are embedded, the rest are in one or more separate documents (**linked** to the previous one)

Extended reference : example

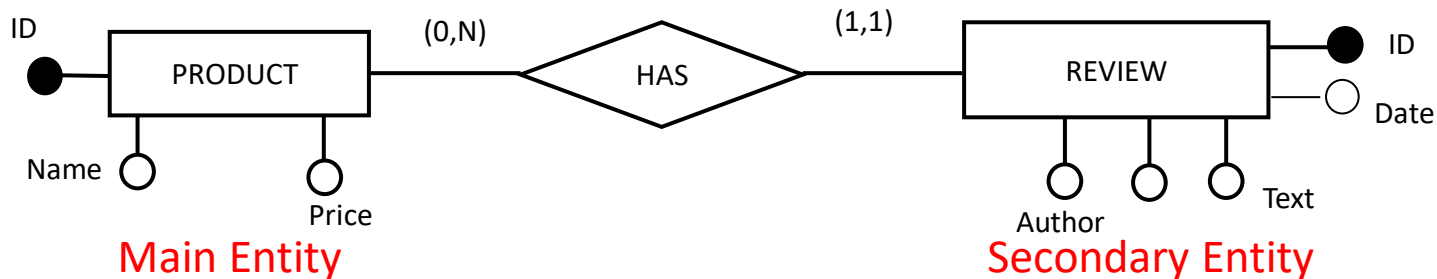


```
{
  _id: 123,
  name: "Katrina Pope",
  street: "123 Main St",
  city: "Somewhere",
  country: "Someplace",
  date_of_birth: ISODate("1992-11-03"),
  social_handles: [
    twitter: "@somethingamazing123"
  ]
  ...
}
```

Nesting: Just a few attributes
+ **Referencing**

```
{
  _id: ObjectId("507f1f77bcf86cd799439011"),
  date: ISODate("2019-02-18"),
  customer_id: 123,
  shipping_address: {
    name: "Katrina Pope",
    street: "123 Main St",
    city: "Somewhere",
    country: "Someplace"
  },
  order: [
    {
      product: "widget",
      qty: 5,
      cost: {
        value: NumberDecimal("11.99"),
        currency: "USD"
      }
    }
  ],
  ...
}
```

Extended reference: example



```
Products = [{
  _id: 1,
  name: "Super Widget",
  price: 119.99,
  reviews: [{
    review_id: 786,
    star: 5},
    {
    review_id: 785,
    star: 4}, ...], ...]
```

```
Reviews = [{
  review_id: 786,
  product_id: 1,
  author: "Kristina",
  text: "An amazing widget.",
  star: 5,
  date: "2019-02-18"},
  {
  review_id: 885,
  product_id: 1,
  author: "Ken",
  text: "Not too good. ",
  star: 5,
  date: "2019-03-17"}, ...]
```

Nesting: Just a few attributes

Two-way Referencing

Extended reference Pattern

Pros

- Faster reads
- Reduce the number of joins

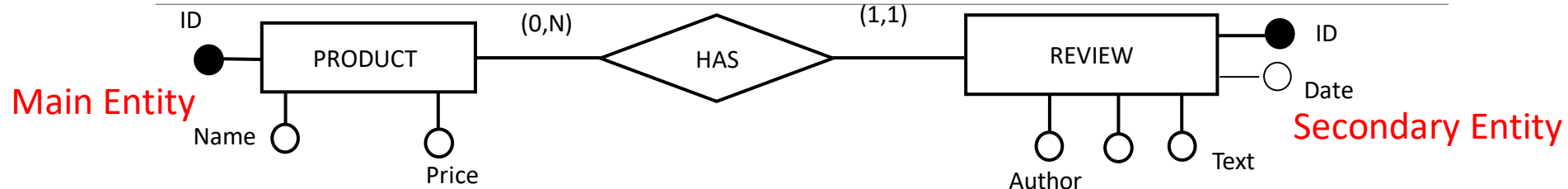
Cons

- It can introduce duplicates and slow down refresh operations if the extended reference contains data that changes a lot.

Guidelines

- Embed fields that don't change often
- Embed fields that needs to be historicized (e.g., order shipping address)
- Include all fields that are needed to avoid joins

Subset Pattern: example



```
products = [ {  
  id: 1,  
  name: "Super Widget",  
  price: 119.99,  
  reviews: [{  
    review_id: 786,  
    author: "Kristina",  
    text: "An amazing widget.",  
    star: 5,  
    date: "2019-02-18"},  
    {  
    review_id: 785,  
    author: "Trina",  
    text: "Nice product. ",  
    star: 4,  
    date: "2019-02-17"} }], ...]
```

```
reviews = [{  
  review_id: 786,  
  product_id: 1,  
  author: "Kristina",  
  text: "An amazing widget.",  
  star: 5,  
  date: "2019-02-18"},  
  {  
    review_id: 885,  
    product_id: 1,  
    author: "Ken",  
    text: "Not too good. ",  
    star: 4,  
    date: "2019-03-17"}, ...]
```

Referencing: All reviews
(in separate documents)

Nesting: Only the last 10 reviews

Outlier Pattern: example

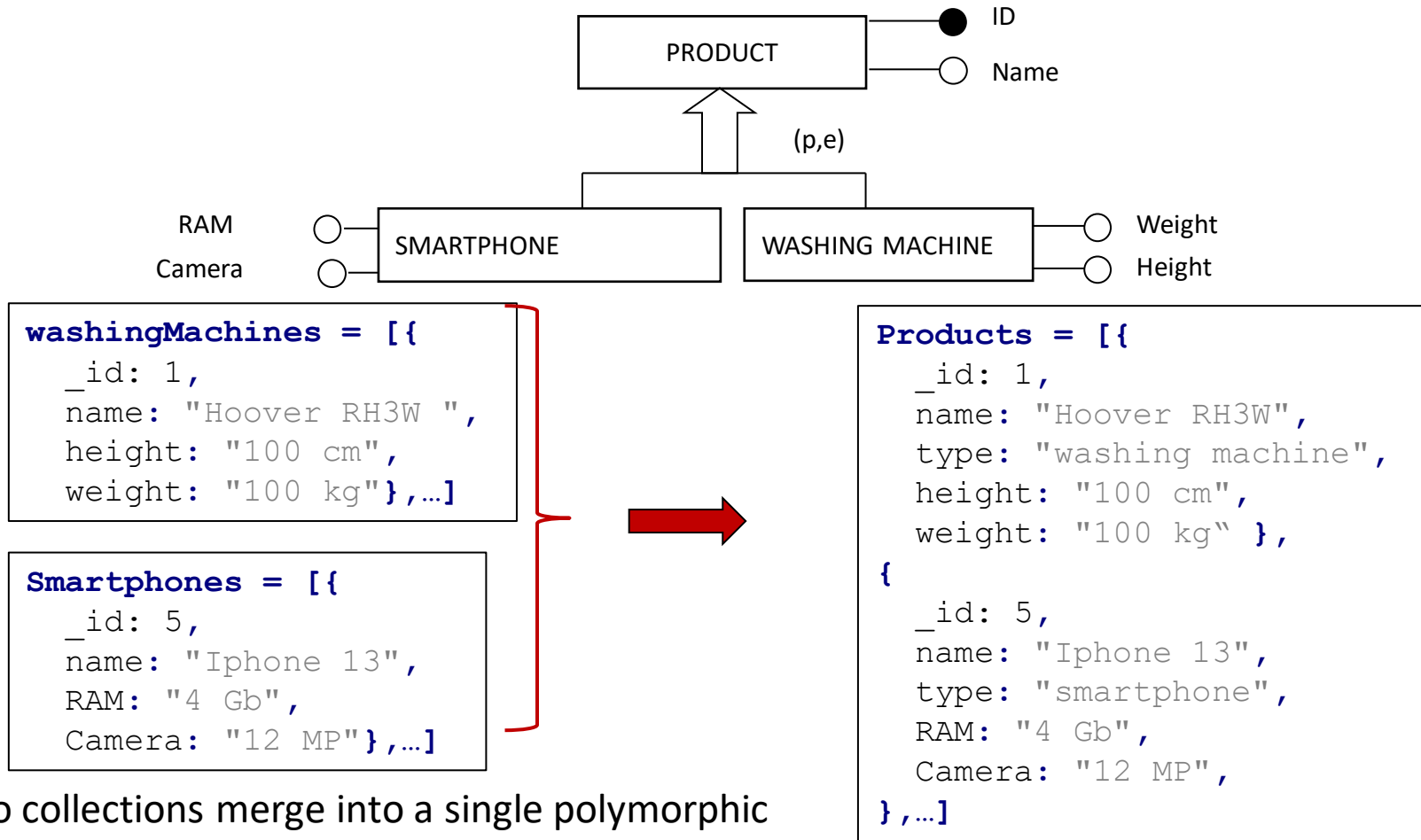
```
products = [ {  
  _id: 1,  
  name: "Super Widget",  
  price: 119.99,  
  reviews: [{  
    review_id: 786,  
    author: "Kristina",  
    text: "An amazing widget.",  
    star: 5,  
    date: "2019-02-18"},  
    {  
    review_id: 787,  
    author: "Trina",  
    text: "Nice product. ",  
    star: 4,  
    date: "2019-02-17" }, ...],  
  has_extras_reviews: 1 } ,...]
```

```
product_reviews = [ {  
  _id: 1111,  
  product_id: 1,  
  page_number: 1,  
  reviews: [{  
    review_id: 788,  
    author: "Ken",  
    text: "Top widget.",  
    star: 5,  
    date: "2019-03-18"},  
    {  
    review_id: 789,  
    author: "Jim",  
    text: "Good product. ",  
    star: 4,  
    date: "2019-02-17" },  
    ...],  
  } ,...]
```

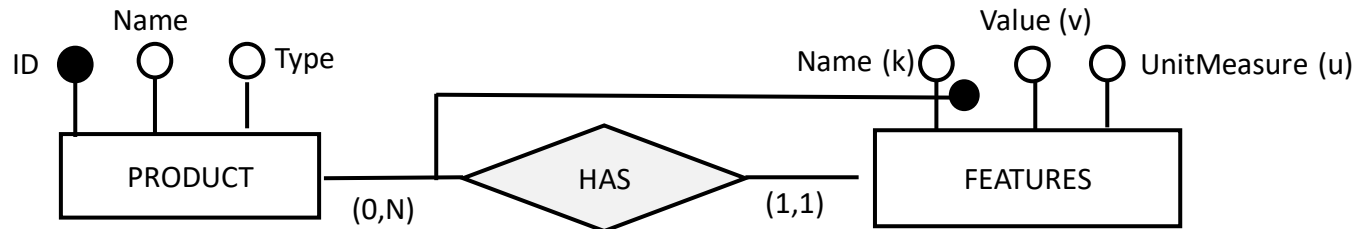
Nesting: up to N reviews
has_extras: outliers flag

Referencing: The remaining reviews of a product are embedded in one or more buckets (paginated documents)

Polimorphic Pattern



Attribute Pattern



```
Products = [{
  _id: 1,
  name: "Hoover RH3W",
  type: "washing machine",
  height: "100 cm",
  weight: "100 kg" },
{
  _id: 5,
  name: "Iphone 13",
  type: "smartphone",
  RAM: "4 Gb",
  Camera: "12 MP",
}, ...]
```

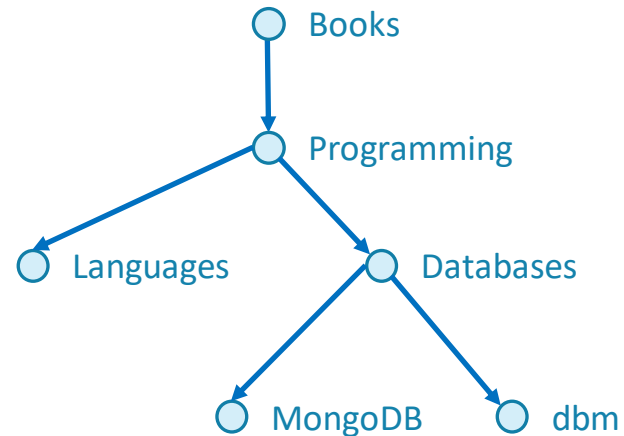
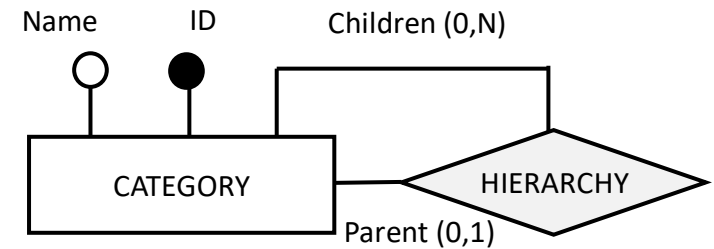
Features are transformed
in an array of key-value pairs

```
Products = [{
  _id : 1,
  name: "Hoover RH3W",
  type: "washing machine",
  specs: [{k: "height",
            v: 100, u: "cm"},
          {k: "weight",
            v: 20, u: "kg"} ]},
{
  _id : 5,
  name: "Iphone 13",
  type: "smartphone",
  specs: [{k: "Ram",
            v: 4, u: "Gb"}, ...]}, ...
}, ...]
```

Tree Pattern

There are several methods:

- Keeping the recursion
 - Method 1: Refer to the Parent
 - Method 2: Child List
- Flattening the recursion
 - Method 3: Array of Ancestors



Tree Pattern

❑ Method 1: Reference to the Parent

```
Categories = [{ _id: "MongoDB", parent: "Databases" },  
{ _id: "dbm", parent: "Databases" },  
{ _id: "Databases", parent: "Programming" },  
{ _id: "Languages", parent: "Programming" },  
{ _id: "Programming", parent: "Books" },  
{ _id: "Books", parent: null }]
```

❑ Method 2: Child List

```
Categories = [{ _id: "MongoDB", children: [] },  
{ _id: "dbm", children: [] },  
{ _id: "Databases", children: [ "MongoDB", "dbm" ] },  
{ _id: "Languages", children: [] },  
{ _id: "Programming", children: [ "Databases", "Languages" ] },  
{ _id: "Books", children: [ "Programming" ] }]
```

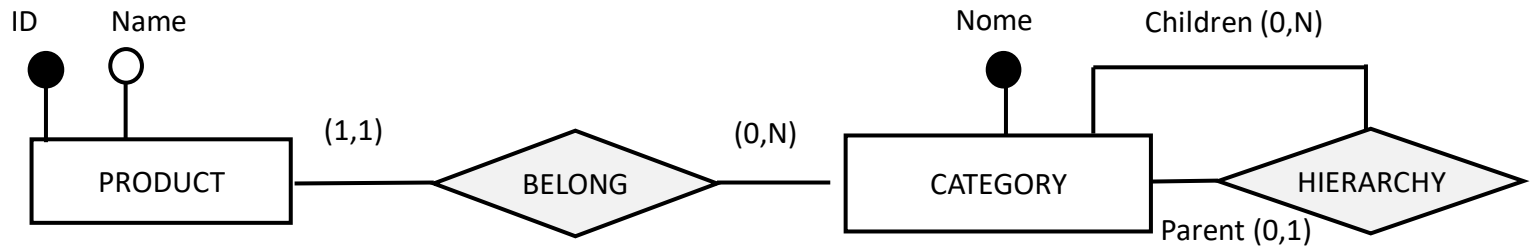
Tree Pattern

❑ Method 3: Array of Ancestors (root-to-node path)

```
Categories = [  
  { _id: "MongoDB", ancestors: [ "Books", "Programming", "Databases" ]},  
  { _id: "dbm", ancestors: [ "Books", "Programming", "Databases" ]},  
  { _id: "Databases", ancestors: [ "Books", "Programming" ]},  
  { _id: "Languages", ancestors: [ "Books", "Programming" ]},  
  { _id: "Programming", ancestors: [ "Books" ]},  
  { _id: "Books", ancestors: [ ]}]
```

The *Array of Ancestors* pattern provides a fast and efficient solution to find the descendants and the ancestors of a node by creating an index on the elements of the ancestors field. This makes *Array of Ancestors* a good choice for working with subtrees.

Array of Ancestors: example



```
Products = {  
  _id: 1,  
  name: "Sams 860",  
  ancestor_categories: ["SSD", "Hard Drive", "Storage", "Computers"]}
```

Store the full path from a node to the top of the hierarchy, as a list of the parents

- data duplication
 - a small cost compared to the benefits you can gain from not calculating the trees all the time.

A Cost Function for Document-DB Workload

Optimizing the design means finding aggregates that minimize the workload cost under reasonable space constraints

- Document DB have a maximum ObjectSize
- The overall DBSize must be defined

The query cost can be estimated as the size in byte of the documents read or written. We assume that:

- It always exists an index on the attributes on which a selection criteria is formulated (but we will not consider the related costs)
- The number of the documents *related* to a primary document through a -to-*n* relationship is estimated as the *average* number given the reported cardinalities
- Document size is given by the sum of its attributes considering embeddings with the average cardinality of the relationships

Collection Size

The average size **of the attributes of each E/R entity** is provided in input, the size of the ids is estimated as a constant

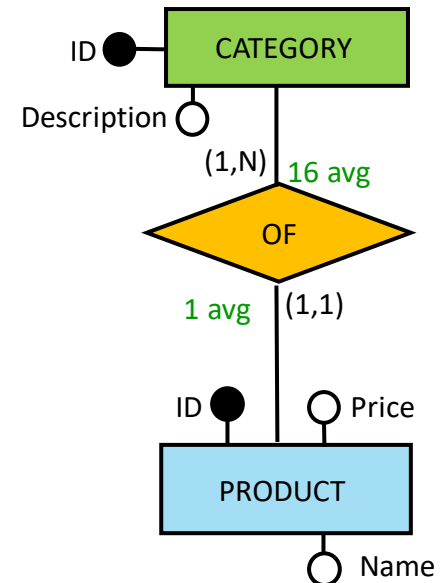
- $len(CATEGORY) = 100$
- $len(PRODUCT) = 135$
- $ND_{CATEGORY} = 50$
- $ND_{PRODUCT} = 800$

$$NP_C = \lceil len(C) \times ND_C / (D \times u) \rceil$$

- $len(id) = 10$
- $D = 4096$ bytes
- $u = 0.69$

```
CATEGORY = {  
  _id: "bev",  
  Description: "beverages"  
}
```

```
PRODUCT = {  
  _id: 1,  
  IdCat: "bev",  
  Name: "Pepsi Cola",  
  Price: 2  
}
```



Collection Size: Reference

- $len(CATEGORY) = 100$
- $len(PRODUCTS) = 135$
- $ND_{CATEGORY} = 50$
- $ND_{PRODUCTS} = 800$
- $len(id) = 10$
- $D = 4096$ bytes
- $u = 0.69$

```
CATEGORY = {
  _id: "bev",
  Description: "beverages"
}
```

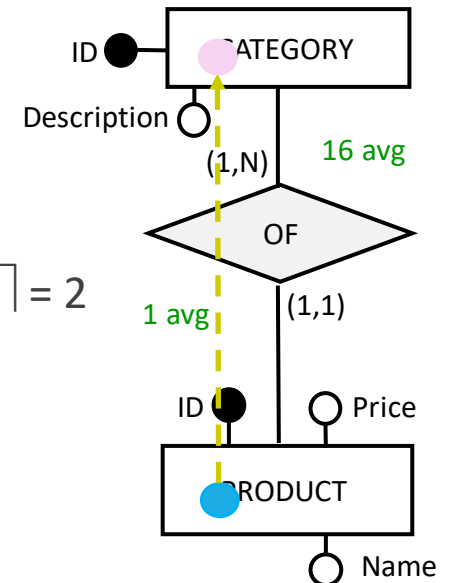
$len(CATEGORY) = 100$

$$NP_{CATEGORY} = \lceil 50 \times 100 / (4096 \times 0.69) \rceil = 2$$

```
PRODUCT = {
  _id: 1,
  IdCat: "bev",
  Name: "Pepsi Cola",
  Price: 2
}
```

$len(PRODUCT) = 135 + 10 = 145$

$$NP_{PRODUCTS} = \lceil 145 \times 800 / (4096 \times 0.69) \rceil = 42$$

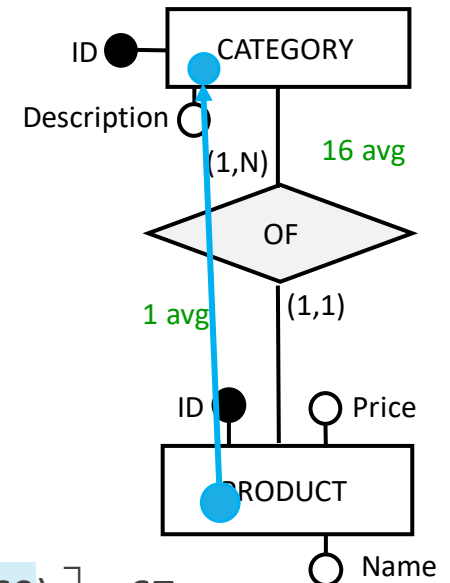


Collection Size: Flattening

- $len(CATEGORY) = 100$
- $len(PRODUCTS) = 135$
- $ND_{CATEGORY} = 50$
- $ND_{PRODUCTS} = 800$
- $len(id) = 10$
- $D = 4096$ bytes
- $u = 0.69$

```
PRODUCT = {
  _id: 1,
  Category : {
    _id: "bev",
    Description: "beverages"
  }
  Name: "Pepsi Cola",
  Price: 2
}
```

$$len(PRODUCT) = 135 + 100 = 235 \quad NP_{PRODUCTS} = \lceil \frac{235 \times 800}{(4096 \times 0.69)} \rceil = 67$$

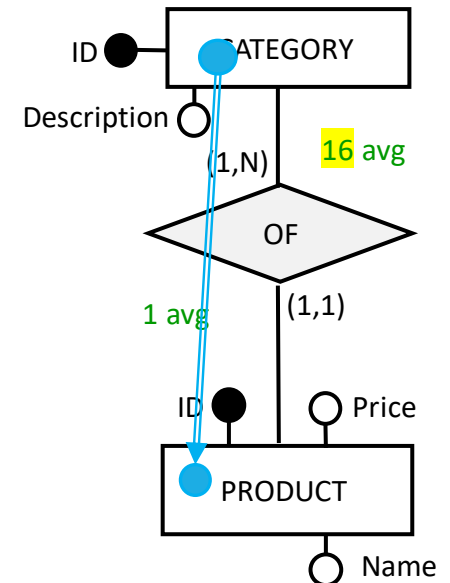


This solution requires more space as it redundantly stores category data for each product

Collection Size: Embedding

- $len(CATEGORY) = 100$
- $len(PRODUCTS) = 135$
- $ND_{CATEGORY} = 50$
- $ND_{PRODUCTS} = 800$
- $len(id) = 10$
- $D = 4096$ bytes
- $u = 0.69$

```
CATEGORY = {
  _id: "bev",
  Description: "beverages"
  Products: [{ _id: 1,
    Name: "Pepsi Cola",
    Price: 2 },
    { _id: 2,
    Name: "Fanta",
    Price: 1.8}]
}
```



$$len(CATEGORY) = 100 + 16 * 135 = 2260$$

$$NP_{CATEGORY} = \lceil \frac{2260 \times 50}{(4096 \times 0.69)} \rceil = 40$$

Collection Size: Mixed

- $\text{len}(\text{CATEGORY}) = 100$
- $\text{len}(\text{PRODUCTS}) = 135$
- $\text{ND}_{\text{CATEGORY}} = 50$
- $\text{ND}_{\text{PRODUCTS}} = 800$
- $\text{len}(\text{id}) = 10$
- $D = 4096$ bytes
- $u = 0.69$

```
PRODUCT = {
  _id: 1,
  Category : {
    _id: "bev",
    Description: "beverages"
  }
  Name: "Pepsi Cola",
  Price: 2
}
```

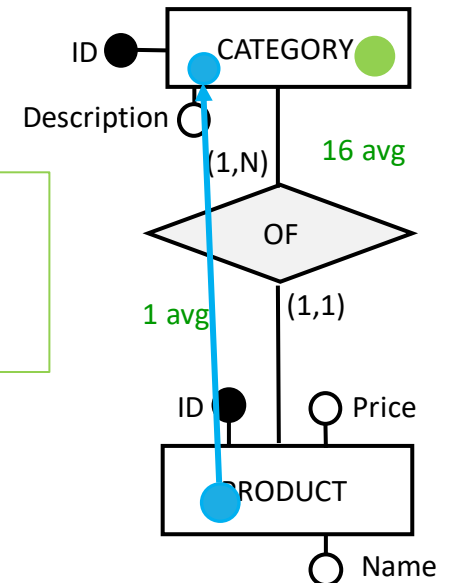
$\text{len}(\text{PRODUCT}) = 235$

$\text{NP}_{\text{PRODUCTS}} = 67$

```
CATEGORY = {
  _id: "bev",
  Description: "beverages"
}
```

$\text{len}(\text{CATEGORY}) = 100$

$\text{NP}_{\text{CATEGORY}} = 2$

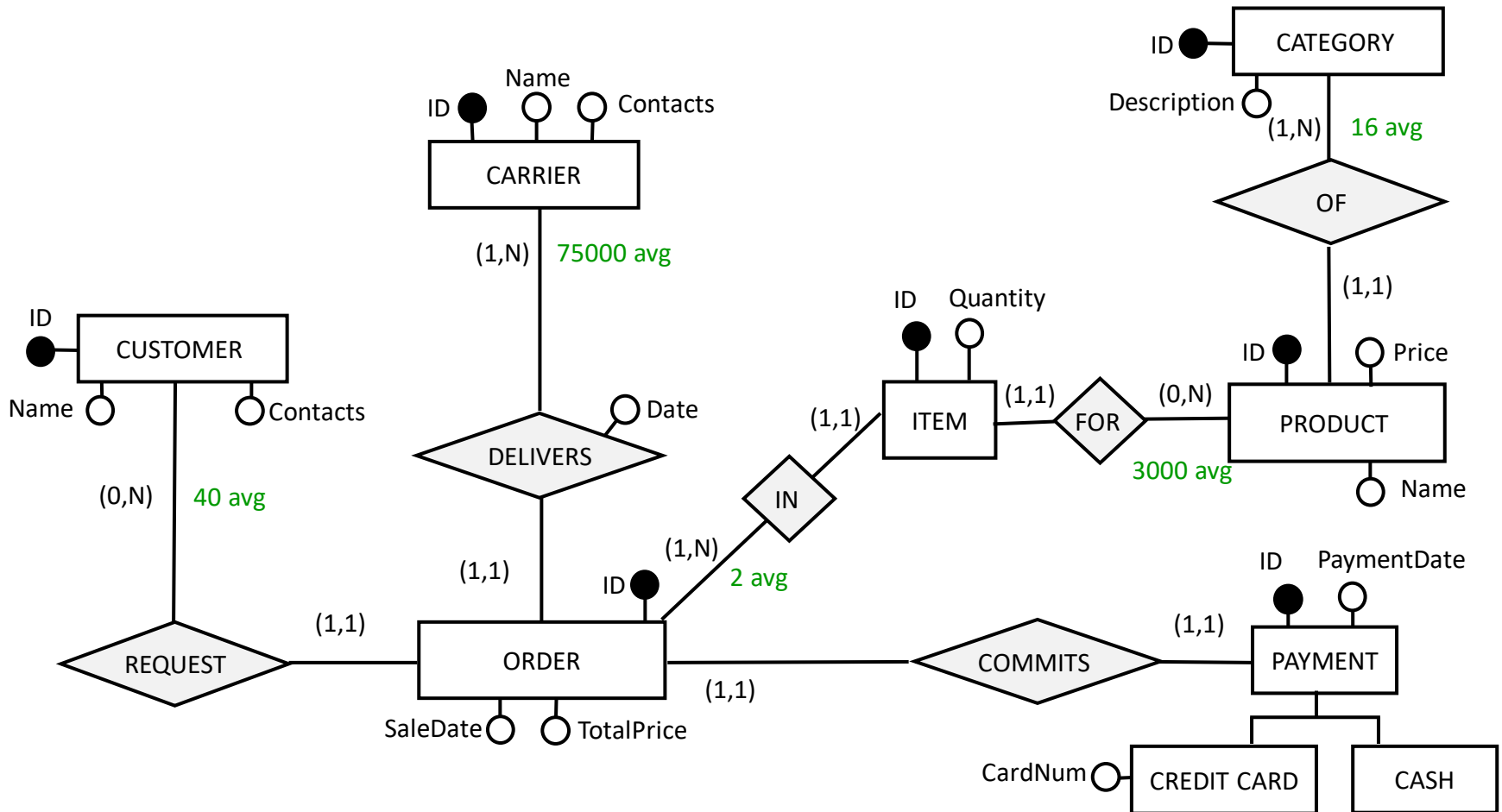


Workload evaluation

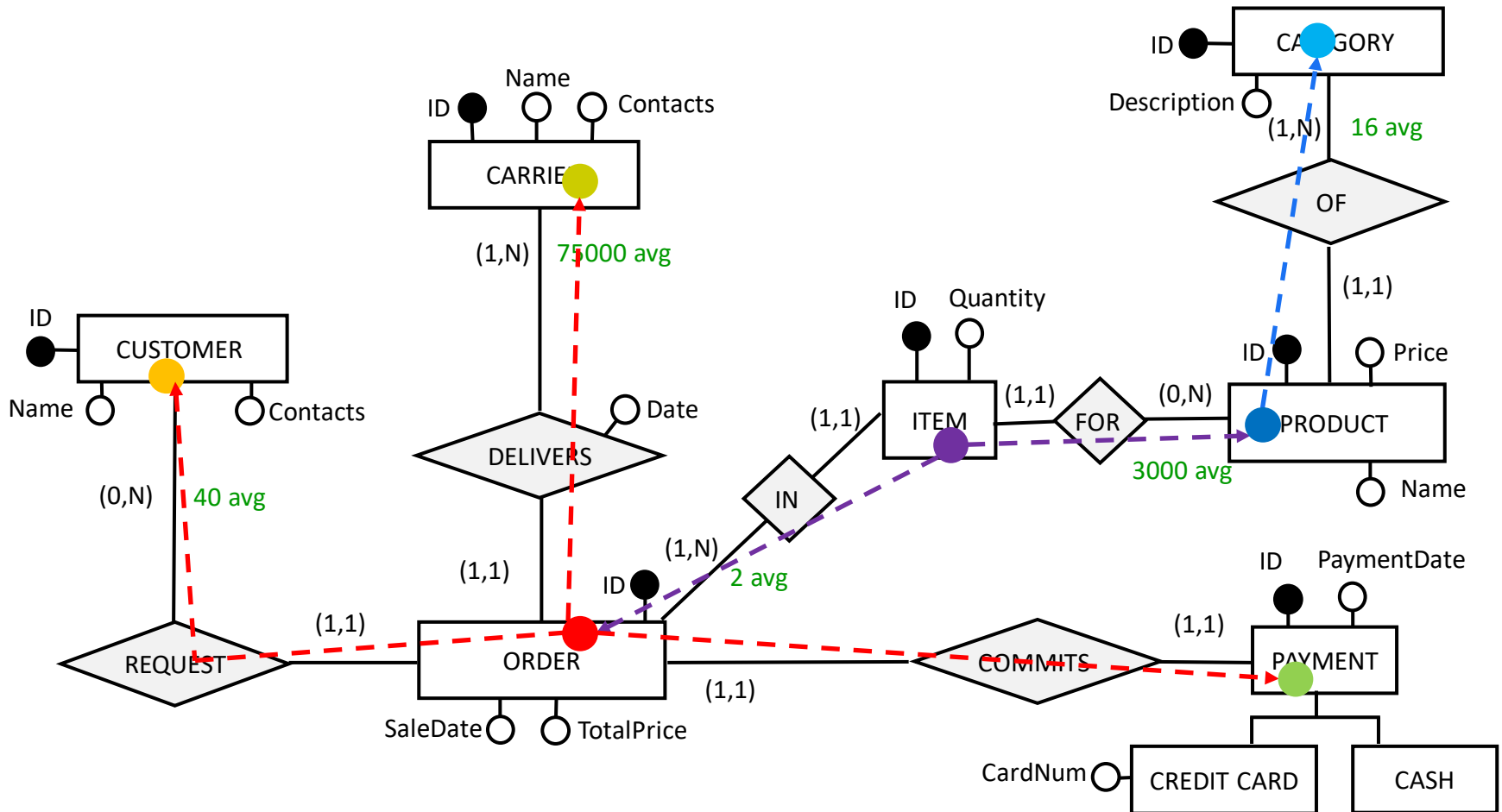
Two solutions are compared within a workload according to the following indicators:

- Workload cost (#documents, #pages)
- The overall size of the collections
- The constraint: each document $< ObjectSize$
- Given the workload:
 - Q1 Given a customerID, select their order details including product name and price
 - Q2 Given and OrderID, select its delivery details
- We want to compare the following solutions:

Case study

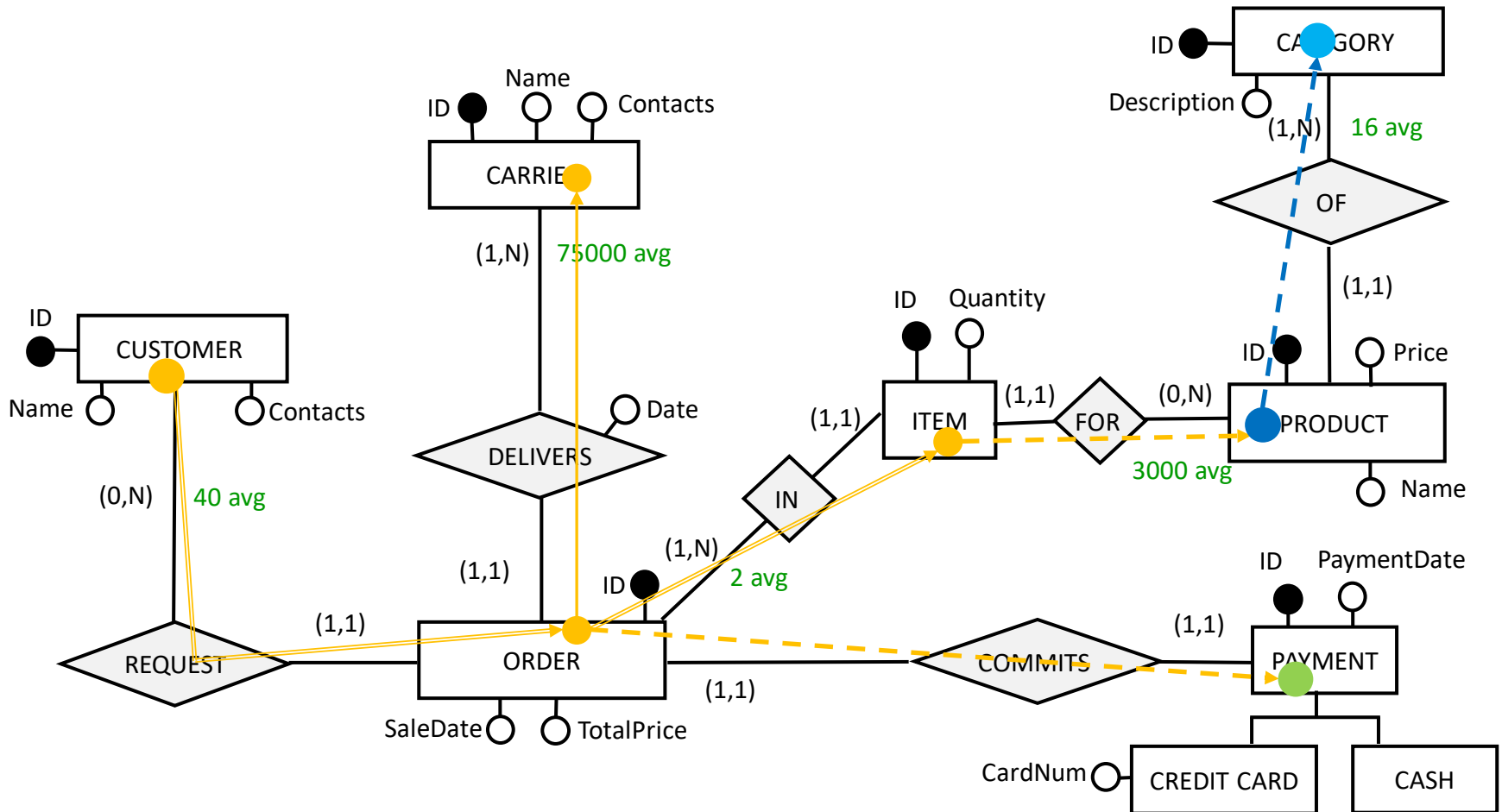


Normalized model



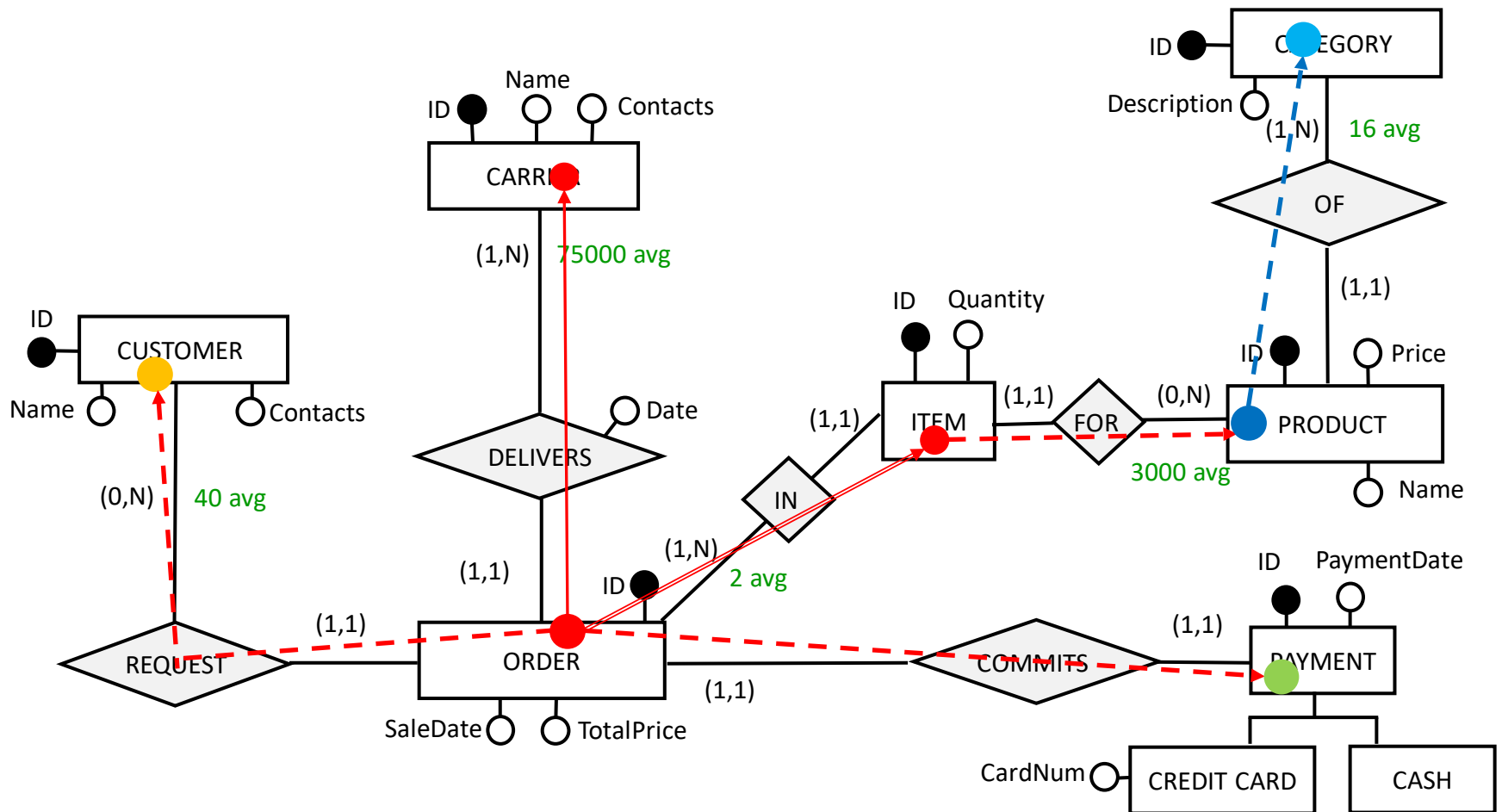
Collections: Category, Payment, Product, Customer, Order, Item, Carrier

Solution 1



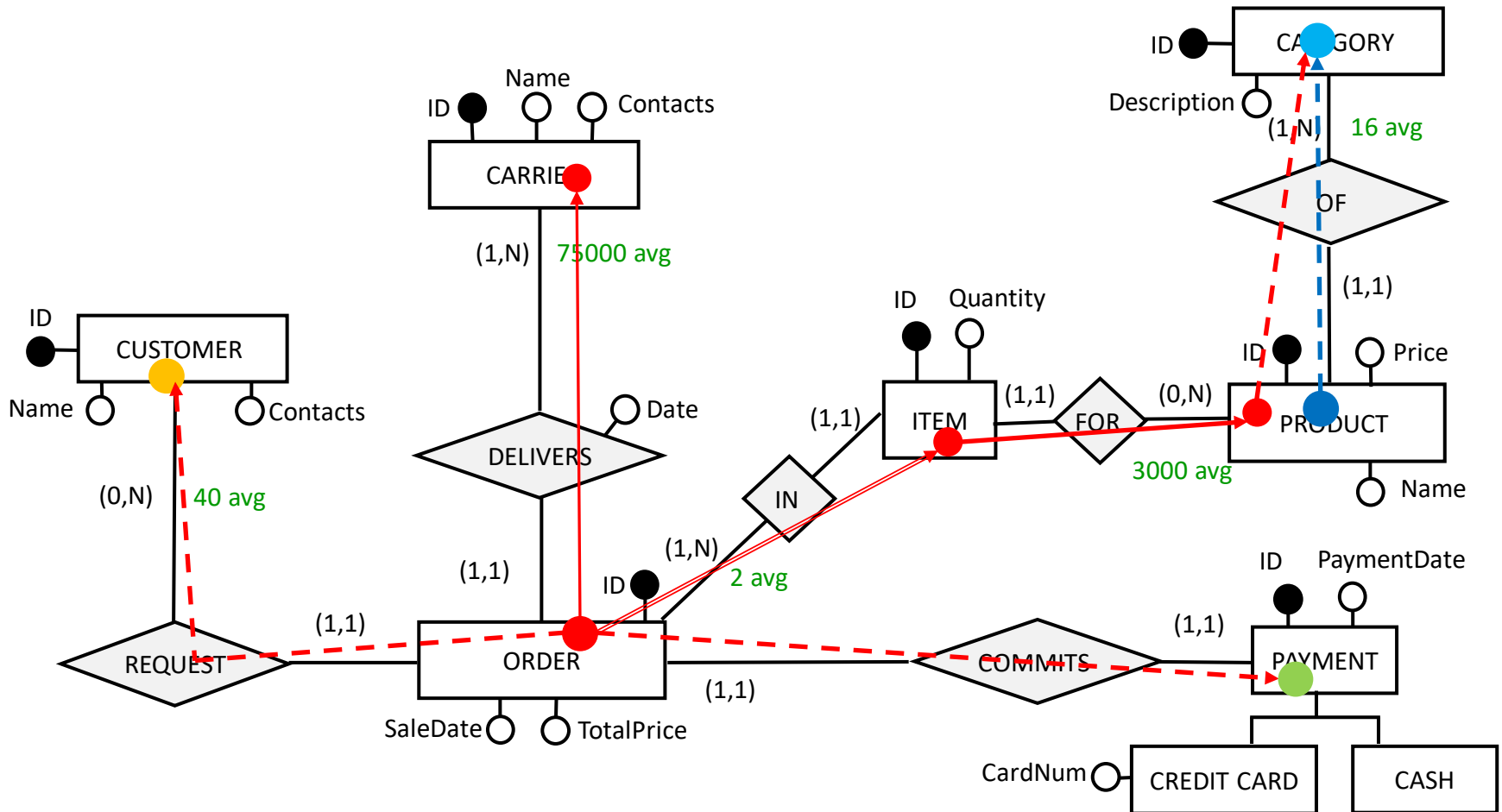
Collections: **Category**, **Payment**, **Product**, **Customer** < (**Order** < (**Item**, **Carrier**))

Solution 2



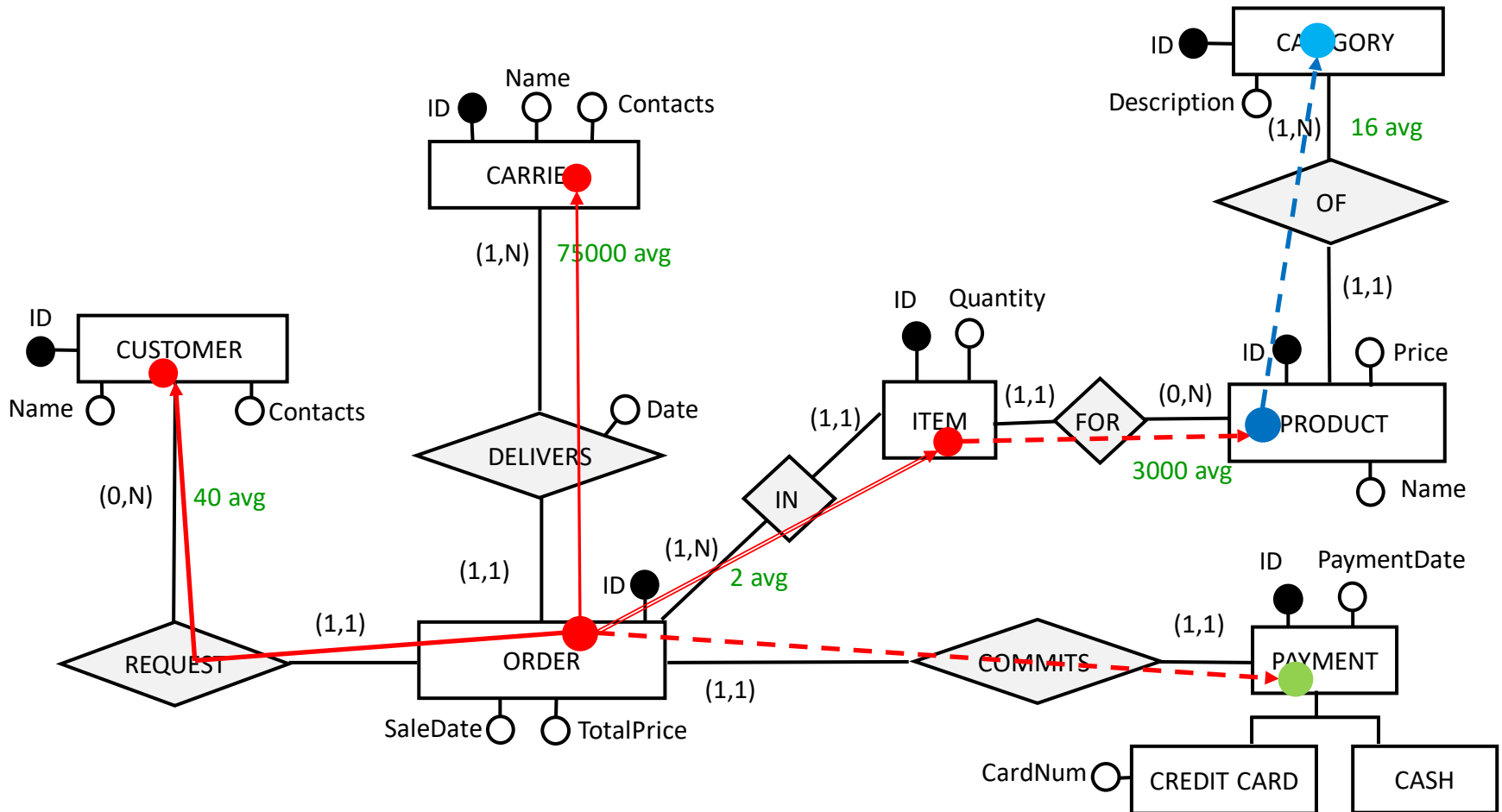
Collections: Category, Payment, Product, Customer, Order < (Item, Carrier)

Solution 3



Collections: Category, Payment, Product, Customer, Order < (Item < Product, Carrier)

Solution 4



Collections: **Category**, **Payment**, **Product**, **Order** < (**Customer**, **Item**, **Carrier**)

Table of Volumes

Name	ND	len
item	2.400.000	100
product	800	135
customer	30.000	125
order	1.200.000	350
carrier	16	120
payment	1.200.000	115
category	50	100

Table of Volumes

SOLUTION 1	len	ND	NP
Category	100	50	2
Payment	115	1200000	48829
Product	145	800	42
Customer	28125*	30000	298542
TOTAL			351466

*Customer takes 10 pages $\lceil 28125 / (4096 \times 0,69) \rceil$

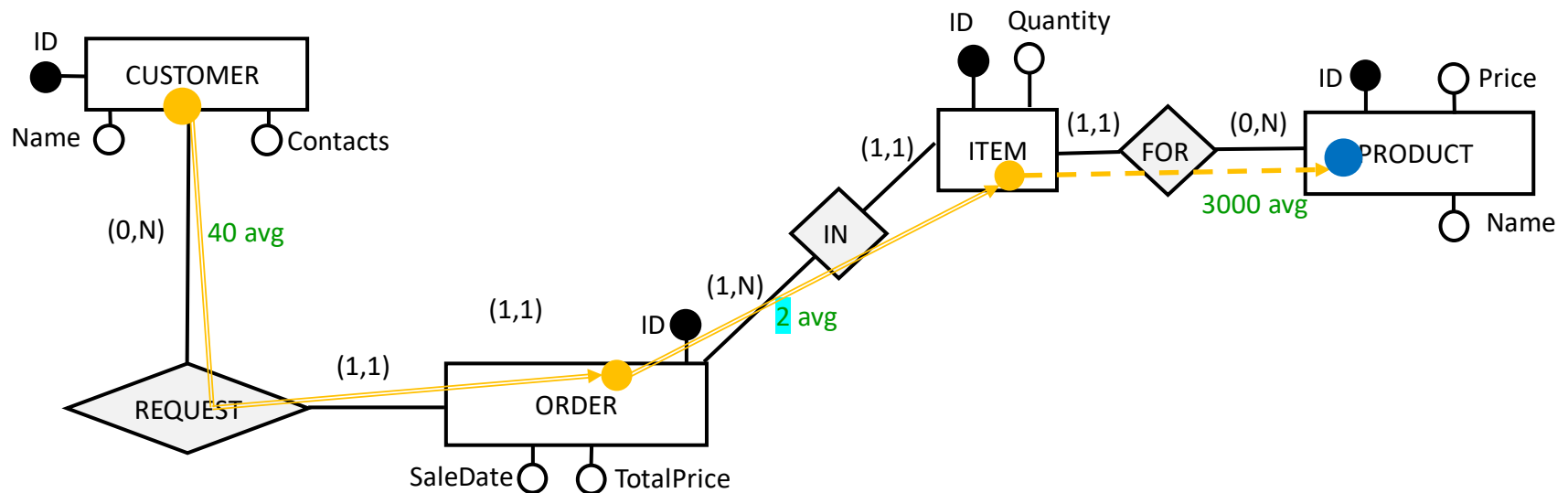
SOLUTION 2	len	ND	NP
Category	100	50	2
Payment	115	1200000	48829
Product	145	800	42
Customer	125	30000	1416
Order	710	1200000	301461
TOTAL			351750

SOLUTION 3	len	ND	NP
Category	100	50	2
Payment	115	1200000	48829
Product	145	800	42
Customer	125	30000	1416
Order	980	1200000	416101
TOTAL			466390

SOLUTION 4	len	ND	NP
Category	100	50	2
Payment	115	1200000	48829
Product	145	800	42
Order	825	1200000	350289
TOTAL			399162

Read cost: Q1 solution 1

Q1 Given a customerID, select their name, orders and details including product name and price

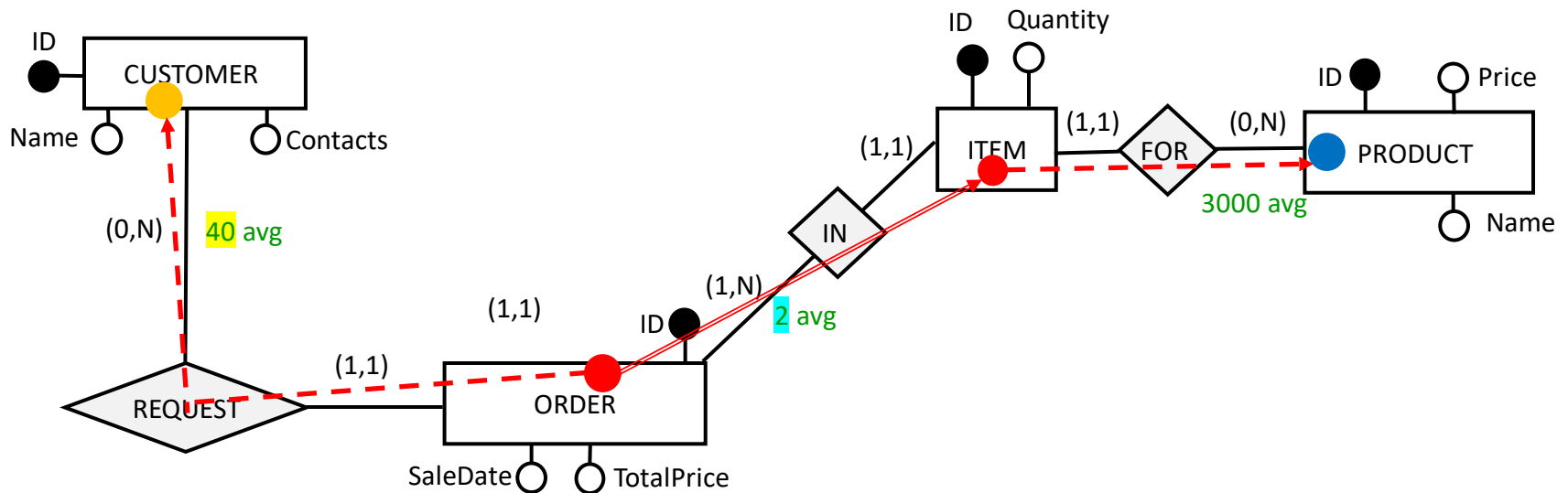


$$ED = ED_{\text{CUSTOMER}} + ED_{\text{PRODUCT}} = 1 + 40 * (2 * 1) = 1 + 80 = 81$$

$$EP = EP_{\text{CUSTOMER}} + EP_{\text{PRODUCT}} = 10 + 80 = 90$$

Read cost: Q1 solution 2

Q1 Given a customerID, select their name, orders and details including product name and price

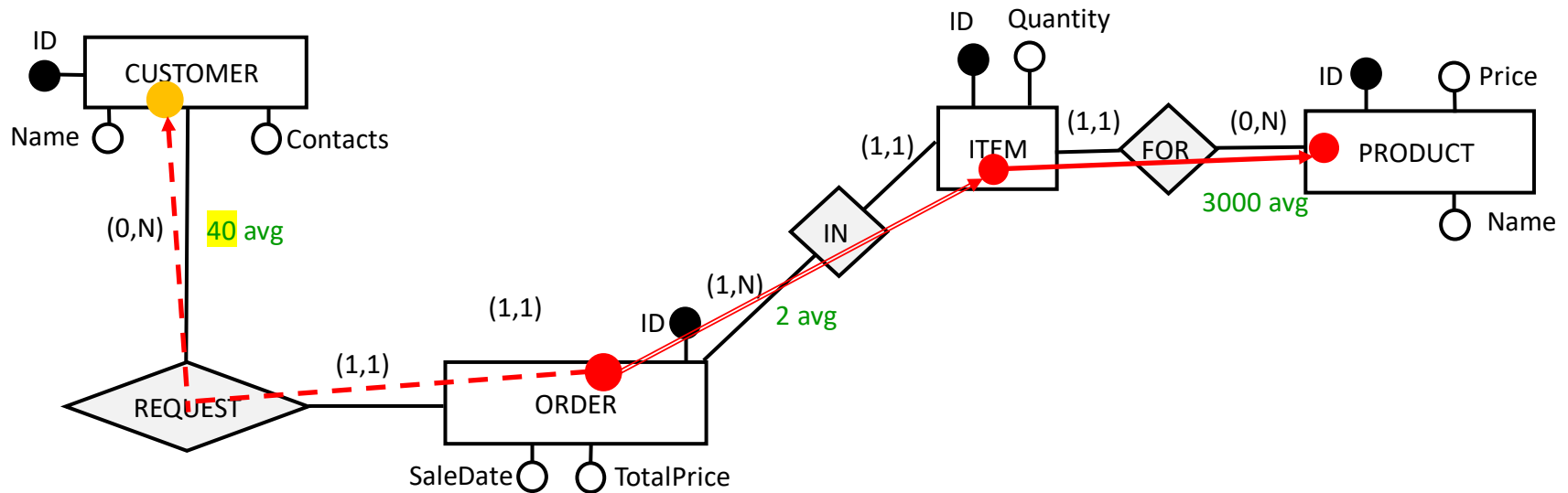


$$ED = ED_{\text{CUSTOMER}} + ED_{\text{ORDER}} + ED_{\text{PRODUCT}} = 1 + 40 + 40 * 2 = 1 + 40 + 80 = 121$$

$$EP = EP_{\text{CUSTOMER}} + EP_{\text{ORDER}} + EP_{\text{PRODUCT}} = 1 + \Phi(ED_{\text{ORDER}}, NP_{\text{ORDER}}) + 80 = 1 + 40 + 80 = 121$$

Read cost: Q1 solution 3

Q1 Given a customerID, select their name, orders and details including product name and price

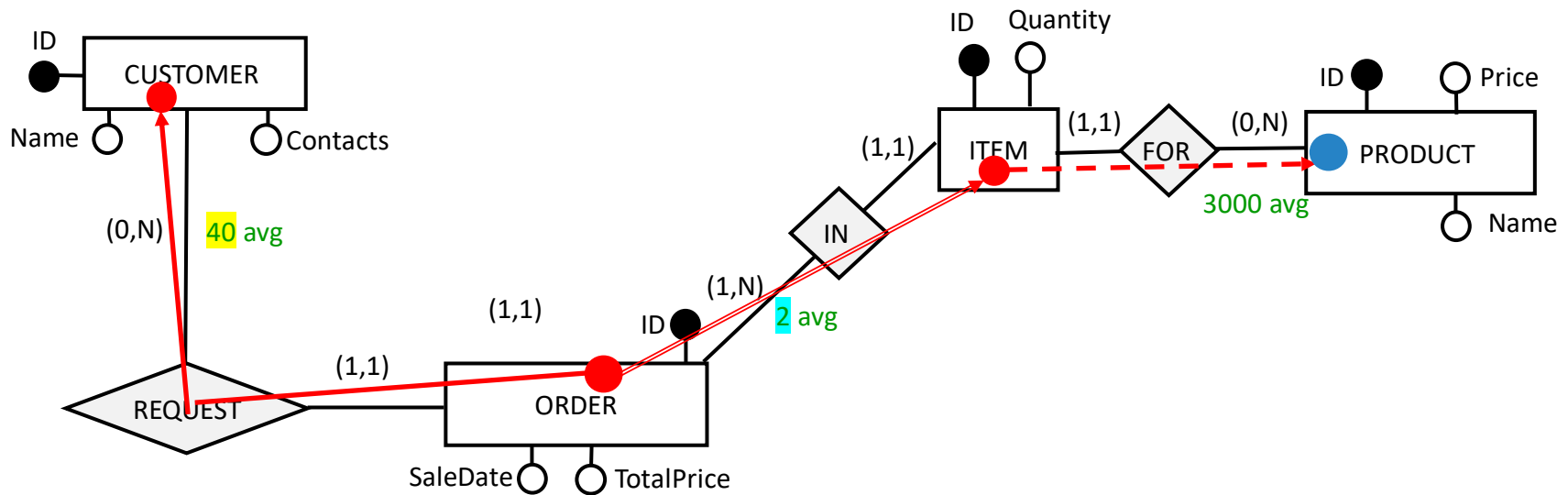


$$ED = ED_{\text{CUSTOMER}} + ED_{\text{ORDER}} = 1 + 40 * 1 = 1 + 40 = 41$$

$$EP = EP_{\text{CUSTOMER}} + EP_{\text{ORDER}} = 1 + \Phi(ED_{\text{ORDER}}, NP_{\text{ORDER}}) = 1 + 40 = 40$$

Read cost: Q1 solution 4

Q1 Given a customerID, select their name, orders and details including product name and price



$$ED = ED_{ORDER} + ED_{PRODUCT} = 40 + 40 * (2 * 1) = 40 + 80 = 120$$

$$EP = EP_{ORDER} + EP_{PRODUCT} = \Phi(ED_{ORDER}, NP_{ORDER}) + 80 = 40 + 80 = 120$$

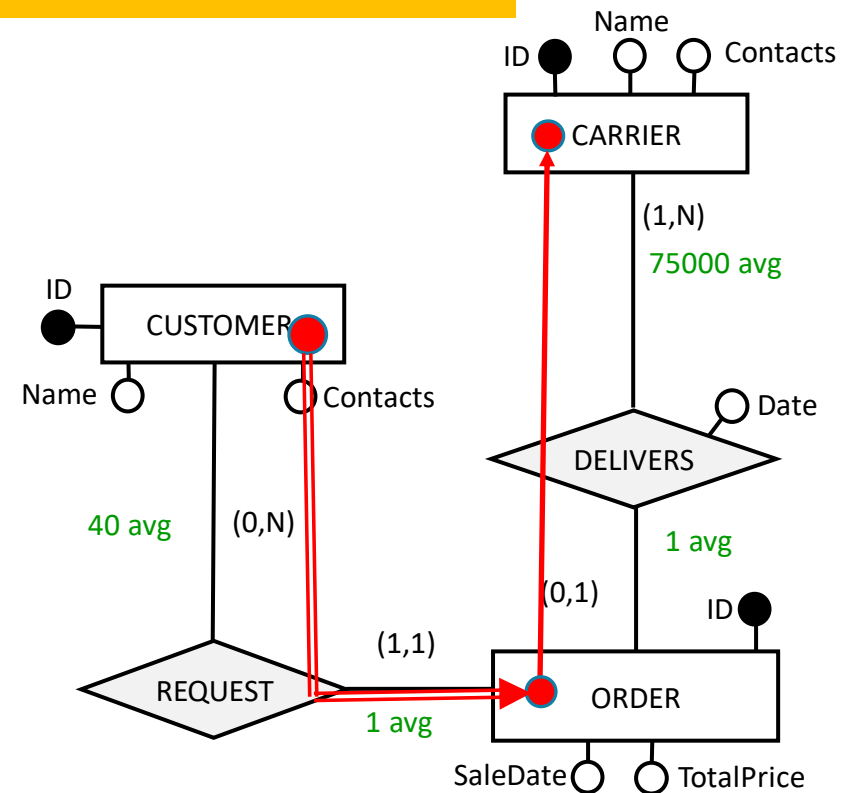
Query cost: Q2 solution 1

Q2 Given and OrderID, select its delivery details

SOLUTION 1

$ED = ED_{\text{CUSTOMER}} = 1$

$EP = EP_{\text{CUSTOMER}} = 10$



Query cost: Q2 solution 2-4

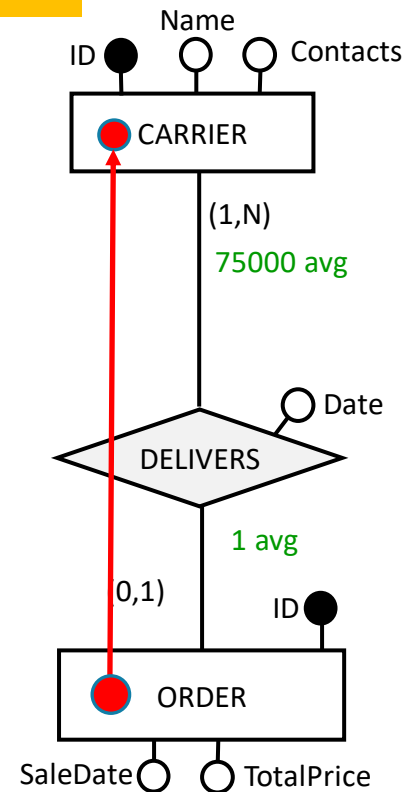
Q2 Given and OrderID, select its delivery details

SOLUTION 2-4

$ED = ED_{ORDER} = 1$

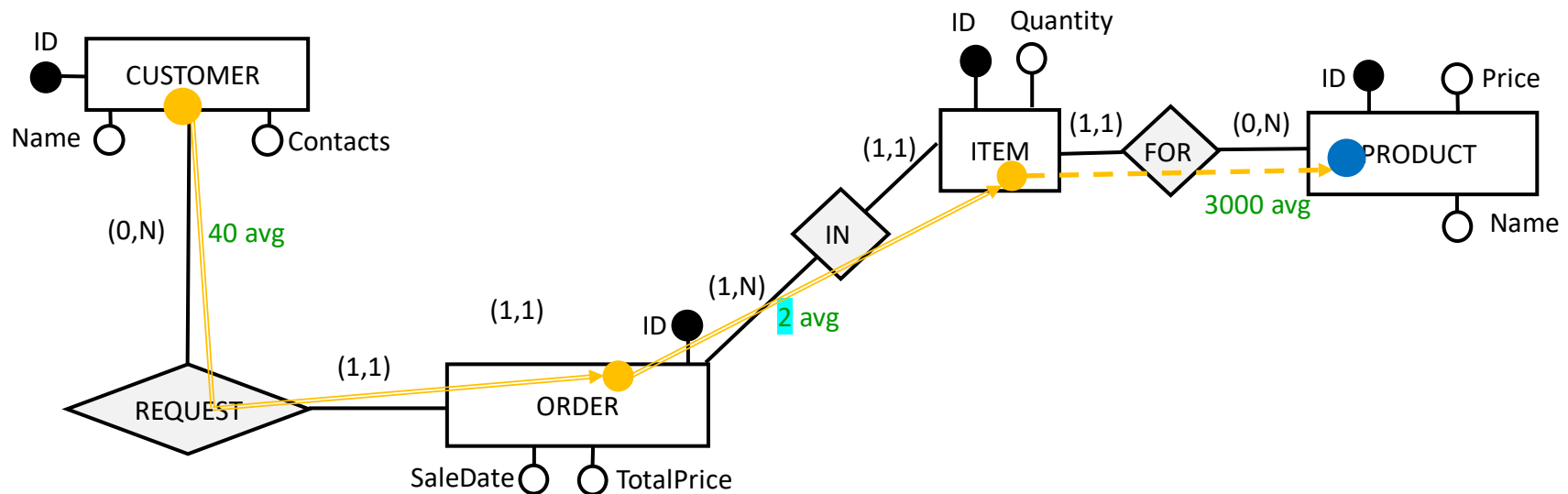
$EP = EP_{ORDER} = 1$

The cost for the solutions 2-4 is the same.
The difference in document size is irrelevant in this case.



Query cost: Q1 solution 1

Q1 Given a customerID, select their name, orders and details including product name and price

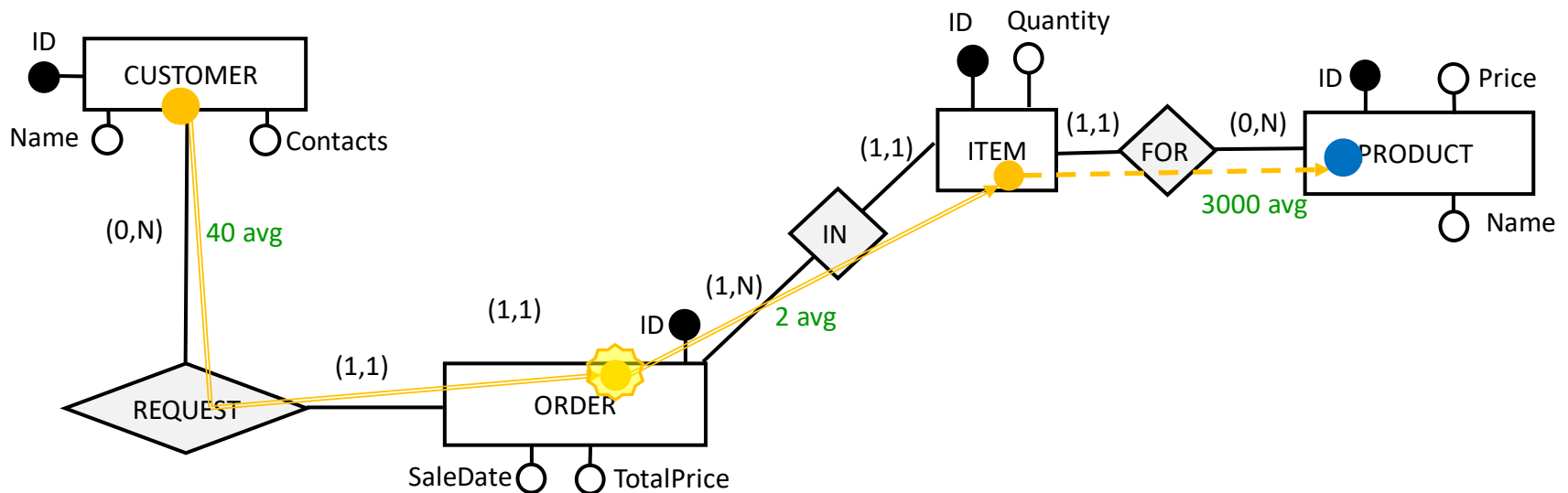


$$ED = ED_{\text{CUSTOMER}} + ED_{\text{PRODUCT}} = 1 + 40 * (2 * 1) = 1 + 80 = 81$$

$$EP = EP_{\text{CUSTOMER}} + EP_{\text{PRODUCT}} = 10 + 80 = 90$$

Write cost: Solution 1

U1: Insert a new Order



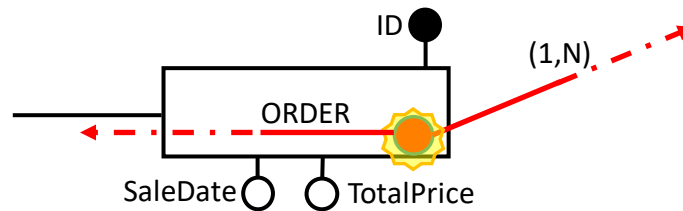
$$ED = ED_{\text{CUSTOMER}} = 1$$

$$EP = EP_{\text{CUSTOMER}} = 10$$

$$\text{TotalCost} = 2 * EP = 20 \text{ (UPDATE)}$$

Write cost: Solution 2-4

U1: Insert a new Order



$$ED = ED_{\text{ORDER}} = 1$$

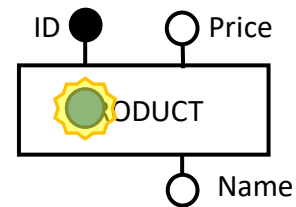
$$EP = EP_{\text{ORDER}} = 1$$

$$\text{TotalCost} = 1 * EP = 1 \text{ (INSERT)}$$

it is assumed that you already know all the data required for input

Write cost: Solution 1,2,4

U2: Update a Product



$$ED = ED_{\text{PRODUCT}} = 1$$

$$EP = EP_{\text{PRODUCT}} = 1$$

$$\text{TotalCost} = 2 * EP = 2 \text{ (UPDATE)}$$

Write cost: Solution 3

U2: Update a Product

$$ED_{\text{PRODUCT}} = 1$$

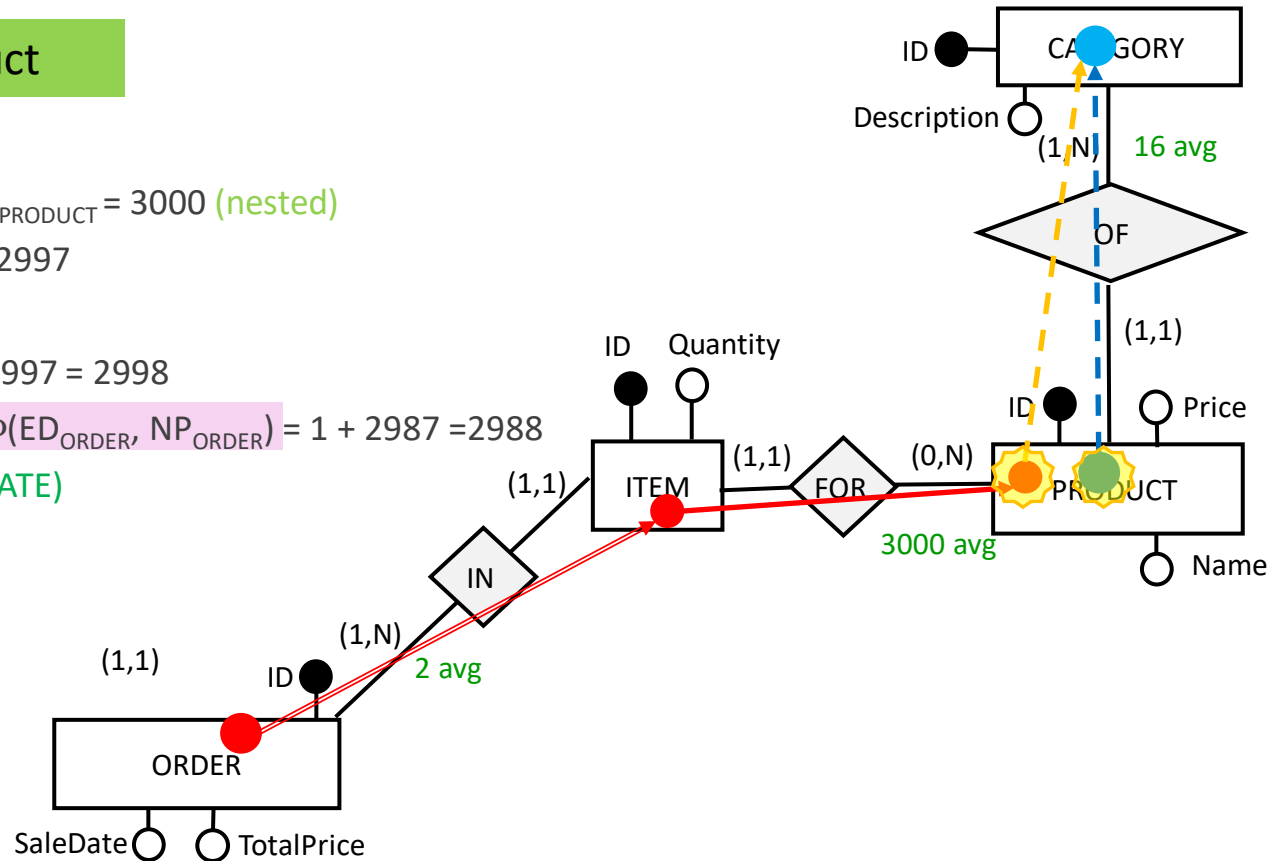
$$ED_{\text{ITEM}} = (ND_{\text{ITEM}} / ND_{\text{PRODUCT}}) ED_{\text{PRODUCT}} = 3000 \text{ (nested)}$$

$$ED_{\text{ORDER}} = \Phi(ED_{\text{ITEM}}, ND_{\text{ORDER}}) = 2997$$

$$ED = ED_{\text{PRODUCT}} + ED_{\text{ORDER}} = 1 + 2997 = 2998$$

$$EP = EP_{\text{PRODUCT}} + EP_{\text{ORDER}} = 1 + \Phi(ED_{\text{ORDER}}, NP_{\text{ORDER}}) = 1 + 2987 = 2988$$

$$\text{TotalCost} = 2 * EP = 5976 \text{ (UPDATE)}$$



Summary (partial)

	Max len	NP	Tot ED	Tot EP
Solution 1	26725	351466	79	106
Solution 2	710	351750	117	117
Solution 3	960	457898	41	41
Solution 4	825	399162	116	116

- Solution 1 has documents with large size
- Solution 3 provides a 66% savings in access costs compared to solution 2, at a 30% increased cost for storage.
- The optimal solution should consider query frequency and update costs.